



Resumen para la I1

Git, estructuras de datos y OOP

Índice

1	Resumen	3
2	Control de versiones (Git)	4
2.1	Mantenion de codigo	4
2.1.1	¿Qué es Git?	4
2.1.2	Git y Github	4
2.2	Instalación de Git	4
2.3	Flujo de control con git	4
2.3.1	Comandos básicos	5
2.3.2	Branches	5
2.4	Github	6
2.4.1	Agregar cuenta	7
2.4.2	Agregar un repositorio remoto	7
2.4.3	Ignorar archivos no deseados (.gitignore)	9
2.5	Clone y Pull Requests	9
2.5.1	Clone	9
2.5.2	Pull requests	10
3	Variables y tipos de datos	12
3.1	Variables, el corazón del código	12
3.2	Tipos básicos de datos	13
3.2.1	Int	13
3.2.2	Float	14
3.2.3	Bool	14
3.2.4	String (str)	15
3.2.5	Resumen	15
3.3	¿Como recibir datos del usuario?	16
3.4	Conversión entre tipos de datos	17
3.5	Ejercicios de práctica	19
3.6	Soluciones a los ejercicios de práctica	20

4	Mas tipos de datos	21
4.1	Listas	21
4.1.1	Acceder a elementos dentro de una lista	21
4.1.2	Métodos útiles de las listas	22
4.1.3	Sliceo de listas	23
4.2	Un poco mas acerca de los Strings	24
4.2.1	Métodos de Strings	24
4.3	Mutabilidad (dato freak)	26
4.4	Tuplas	27
4.4.1	Métodos de tuplas	27
4.5	Sets	28
4.5.1	Métodos de sets	28
4.6	Diccionarios	31
4.6.1	Acceder a los elementos de un diccionario	32
4.6.2	Métodos de diccionarios	33
4.7	Stacks y Colas	35
4.7.1	¿Cómo hago una tupla o un stack en Python?	35
4.8	Tabla comparativa estructuras de datos	36
4.9	Named Tuples	37
4.10	Ejercicios de práctica	38
4.11	Soluciones a los ejercicios de práctica	39
5	Funciones: ¡No repitas código!	42
5.1	¿Qué es una función?	42
5.2	Sintaxis y uso	42
5.3	Argumentos y parámetros	43
5.4	Valores de retorno	45
5.4.1	¿Y si no quiero devolver nada?	45
5.5	*Args y **Kwargs (Avanzado)	46
5.6	Scope de una función (avanzado)	49
5.7	Documentación de funciones (avanzado)	55
5.7.1	Type Hinting	55
5.7.2	Docstrings	56
5.8	Funciones lambda (avanzado)	58
5.9	Decoradores (@) (avanzado)	61
5.10	Ejercicios de práctica	66
5.11	Soluciones a los ejercicios de práctica	67
6	Programación orientada a objetos (OOP)	69
6.1	¿Que es la programación orientada a objetos?	69
6.2	Como aplicar OOP en Python	69
6.2.1	¿Que es una clase?	69
6.2.2	Clases v/s instancias	71
6.2.3	El parámetro "self" y la función __init__()	73
6.3	¿Como usar las instancias?	77

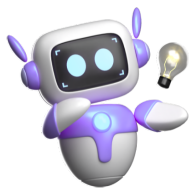
6.4	Dunder methods	80
6.5	Properties (avanzado)	83
6.6	Polimorfismo (avanzado)	88
6.7	Herencia (avanzado)	89
6.7.1	Multiherencia (avanzado)	94
6.7.2	El problema del diamante en Multiherencia	97
6.8	Clases abstractas	101
6.9	Utilizacion de clases abstractas (modulo abc)	102
6.10	Resumen de OOP	105
6.10.1	Cuando usarlo?	105
6.10.2	Clases	105
6.10.3	Instancias	106
6.10.4	Herencia (avanzado)	106
6.10.5	Clases abstractas (modulo abc)	107
6.11	Ejercicios de práctica	107
6.12	Soluciones a los ejercicios de práctica	109

1. Resumen

Esta guía se hizo con el fin de repasar teóricamente los contenidos del ramo antes del midterm. Los contenidos aquí listados fueron extraídos de **Handbook de Python 2025-2**, el cual se encuentra en **Google Drive**:

<https://drive.google.com/drive/folders/1oPTiGMkzIzByE8-6VpXXj0XM8LTWkftd?usp=sharing> junto a mas material de estudio.

Esta guía se compone de varios capítulos, cada uno trata de un tema diferente, pero que de vez en cuando requiere saber algo de los anteriores. Sin embargo, siéntete libre de ir a tu propio ritmo y orden. Todos tenemos distintas bases o distintas maneras de aprender un mismo tema. También, cabe destacar que es imposible abordar ciertos temas sin antes abstraer un poco las cosas más avanzadas, es decir: Hay temas que primero se tratan "por encima" y luego se profundizan en capítulos posteriores, ya que sus aspectos técnicos requieren conocimientos más avanzados dentro de la guía.



Cada capítulo se compone de:

- Materia (tema abordado y principales definiciones)
- Algún que otro tip o dato freak
- **Ejercicios** de práctica

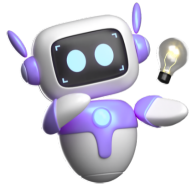
2. Control de versiones (Git)

2.1. Mantencion de codigo

¿Te has preguntado alguna vez como le hacen las grandes empresas para mantener codigos gigantescos en donde colaboran muchas personas a la vez sin que explote nada?

O bien, imagina que tu codigo funciona bien, intentas agregarle algo nuevo y se rompe, ¿hay alguna manera rapida de recuperar una version antes de la catastrofe?

La respuesta es **SI**, hay varias maneras de mantener un codigo a largo y corto plazo, sin embargo, usar **Git** como sistema de control de versiones es la más popular.



Control de versiones

Es una técnica para poder gestionar todos los cambios realizados a un código fuente a lo largo del tiempo. Permite volver a versiones anteriores, trabajar en equipo y mantener un flujo claro de desarrollo.

2.1.1. ¿Qué es Git?

Git es un sistema de control de versiones distribuido, es conocido por ser muy rápido y eficiente. Guarda **snapshots** de todo tu trabajo según necesites. Para posteriormente comparar cambios, recuperar versiones anteriores, etc.

2.1.2. Git y Github

Git es la herramienta que maneja versiones de forma local en tu computador. En cambio, Github es una plataforma online que permite crear y compartir proyectos (repositorios), manteniendo su control de versiones también con Git. Pero permite el respaldo en la nube y el trabajo colaborativo.

2.2. Instalación de Git

Puedes encontrar la documentación completa de como descargar Git en el siguiente enlace:
<https://git-scm.com/downloads>

2.3. Flujo de control con git

El flujo de control en git consta de 4 etapas:

1. Hacer cambios en tu proyecto
2. Agregarlos al "stage area" para quedar guardados en un futuro **snapshot** del proyecto.
3. Cuando estes listo con los cambios, crear un **snapshot** que guarde todo el proyecto en su version actual

A estos **snapshots** se les llama **commits** y son la base de como funciona git. Mientras que el **"staging area"** es un placeholder temporal para los cambios que quieres que incluya el commit.

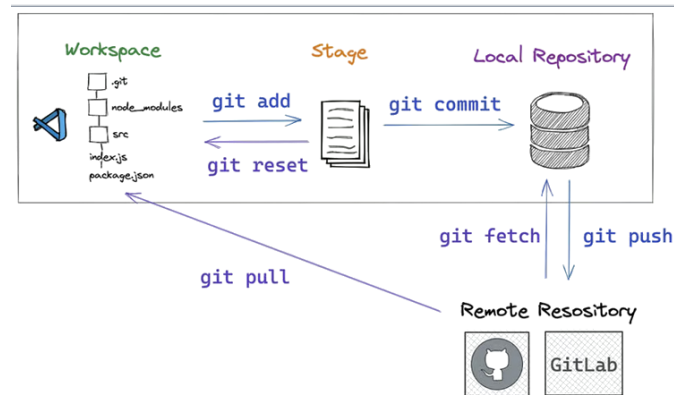
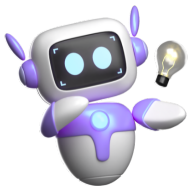


Figura 1: Ejemplo de flujo en Git (Russell Bateman — javahotchocolate.com)

2.3.1. Comandos básicos



- `git init` $\Rightarrow$ Inicializa git (nuevo repositorio)
- `git status` $\Rightarrow$ Muestra el estado actual de tus archivos (cosas cambiadas desde la ultima version)
- `git add 'archivo'` $\Rightarrow$ Agrega archivos al staging area (un atajo para agregar todos los cambios desde la ultima version es colocar `git add .`)
- `git commit -m "mensaje"` $\Rightarrow$ Genera un nuevo commit con "mensaje" como indicador de que cambios se hicieron (es obligatorio agregarle un mensaje descriptivo a cada commit)
- `git log` $\Rightarrow$ Muestra todos los commits desde que se inicializo el repositorio hasta ahora

2.3.2. Branches

Que pasa si quieres hacer multiples cambios en el codigo principal, ya sea tu solo o entre varias personas?

Aqui es donde entran las branches, son basicamente universos paralelos creados a partir de un commit en especifico de la **main branch**, aqui puedes hacer todo lo que quieras y luego simple-

mente **fusionarlo (merge)** con la principal cuando estes listo. Asi puedes trabajar en multiples cambios a la vez sin que interfieran entre si.

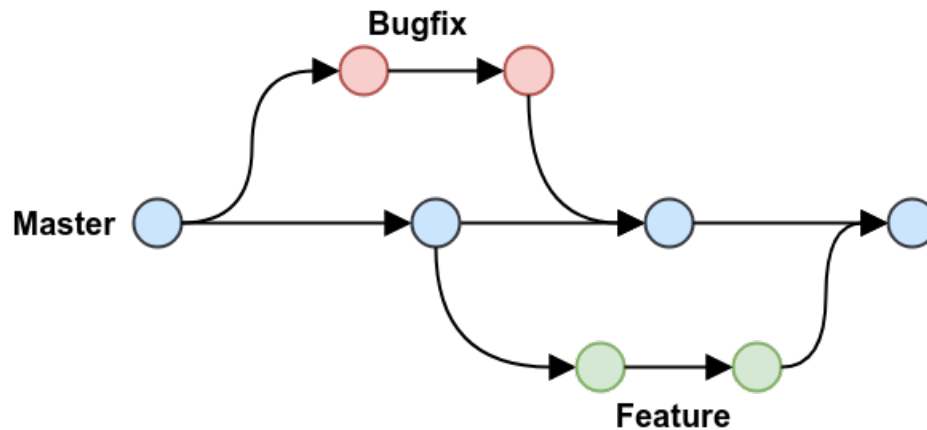
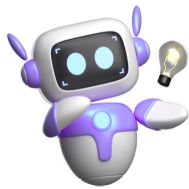


Figura 2: Ejemplo de como funcionan las branches (Github.com)



Comandos con branches

- `git branch nombre_rama` $\Rightarrow$ Crea una nueva rama desde el commit actual, llamada "nombre_rama"
- `git checkout nombre_rama` $\Rightarrow$ Cambia el flujo de trabajo actual a esa rama, todos los cambios posteriores serán guardados en ella.
- `git merge rama_1 rama_2` $\Rightarrow$ Fusiona una rama con otra

¡Cuidado!

Algunas veces pasara que hay ramas que interfieren entre si con sus cambios, estos son los llamados **merge conflicts**. No hay una manera facil de lidiar con ellos mas que hacer cambios a mano para que calzen entre si, asi que trata de mantener un flujo de trabajo ordenado para que no te pasen!

PD: Para saber mas facil y visualmente que diferencias hay en un archivo entre una rama y otra puedes usar el comando `git diff 'nombre_archivo'`



2.4. Github

Como se mencionó anteriormente, existe una plataforma online colaborativa llamada Github para que puedas compartir tus proyectos o basarte en otros!

PD: Este es como el "Facebook" de los programadores :D



Figura 3: Logo de GitHub

2.4.1. Agregar cuenta

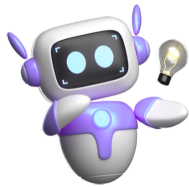
Primero, hay que tener identidad, sino.. git no sabra quien esta haciendo que cambio.

1. Crea una cuenta en **github.com**
2. Configura Git con tus credenciales:

- `git config --global user.name "TuNombre"`
- `git config --global user.email "tu@email.com"`

2.4.2. Agregar un repositorio remoto

Para agregar repositorios remotos en nuestra version local de git tenemos los siguientes comandos:



- `git remote add 'nombre_rama_remota' 'url'`: Agrega a tu repositorio una rama especial llamada "**nombre_rama_remota**" que interconecta tus cambios con los del repositorio online encontrado en "**url**". (por lo general se le llama **origin**)
- `git push ramaRemota ramaLocal`: Agrega al repositorio remoto una rama local, si ya existe, hace **merge** con sus cambios.
- `git fetch`: Funciona como **status** pero lo hace comparando el estado actual de tu repositorio local versus el repositorio remoto, reportando las diferencias que hayan entre si. (te dira que archivos locales estan desactualizados o actualizados con respecto a los que estan en la version subida a github).
- `git pull`: Es una combinacion entre **fetch** y **merge**. Su funcion es actualizar tu version local a lo que sea que es el estado actual de la remota, esto para que esten a la par con los archivos. (en realidad casi nadie usa fetch y merge por separado y se usa este comando que hace un 2x1)

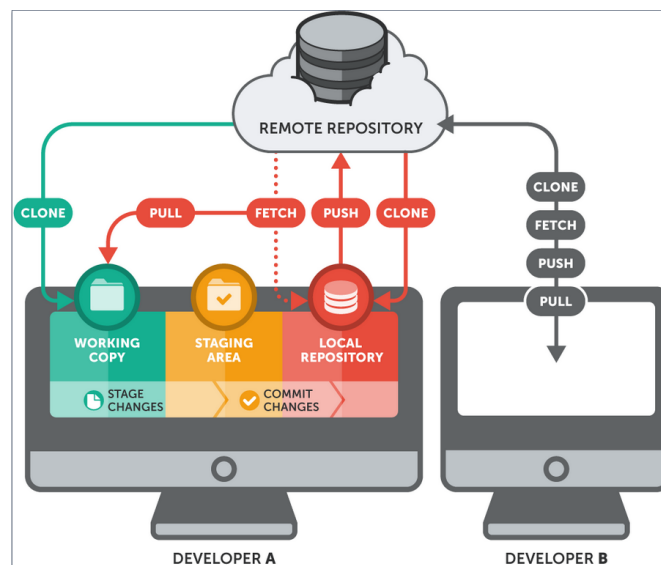


Figura 4: Diagrama de funcionamiento de Git para repositorios remotos (git-tower.com)

¡Cuidado!

Un detalle importante a la hora de pushear cambios a tu repositorio remoto es que si no has hecho un **pull** antes. Los cambios no estaran acorde al estado actual de los archivos y te dara un **merge conflict** gratuito. Asi que siempre antes de hacer cambios en tu version local asegurate de hacer un **pull** de tu version remota para estar al dia con los cambios!



2.4.3. Ignorar archivos no deseados (.gitignore)

Tambien, puede pasar que no quieras subir TODOS los archivos de tu repositorio local (contraseñas, datos sensibles, etc.) a tu repositorio remoto (GitHub). Para ello, puedes crear un archivo llamado **.gitignore** que contiene todos los nombres de archivos que no quieres que sean incluidos cuando agregas todo al "staging area" (`git add .`).

El archivo tambien tiene algunos atajos y tips de formato para ahorrar datos/nombres redundantes de archivos (carpetas completas, tipos de archivo similares, etc.). Aqui te dejo una guia completa con estos "**shortcuts**".

2.5. Clone y Pull Requests

GitHub permite la colaboración entre varias personas, sin embargo, sería un caos si todo el mundo pudiera modificar el código de todo el mundo. Para lo cual existe el poder de clonar un repositorio sin afectar a la copia original. A esta copia se le llama **fork**.

Esto es útil a la hora de querer aportar en proyectos más grandes y hechos por otros programadores sin afectar su flujo principal directamente, con la copia del repositorio puedes probar lo que tú quieras y luego si quieres que el dueño del repositorio vea tus cambios y posiblemente los agregue, a esto se le llama **pull request**.

En resumen:

- **Fork:** Tu versión copiada del proyecto original
- **Pull request:** Una sugerencia al dueño original para que vea y agregue tus cambios al proyecto original.

2.5.1. Clone

Para hacer un fork a un proyecto, se usa el comando **clone**:

```
git clone 'url del repositorio'
```

Pero... como la conseguimos?, al abrir el repositorio que quieres clonar, en el apartado que dice "code" veras un submenu que te dara un link para ejecutar el comando clone.

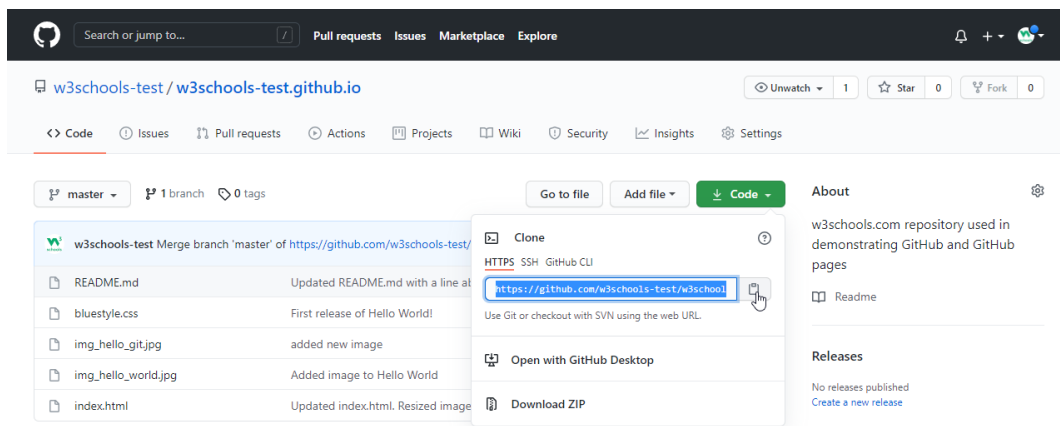


Figura 5: Ejemplo obtencion de url repositorio para clonar (w3schools.com)

2.5.2. Pull requests

Para crear un **pull request** solo basta con hacerle cambios a tu fork y luego hacerles **push** como vimos en la seccion anterior. Luego, al dueño original se le notificara que cambios hiciste y que los quieres fusionar con su "main branch".

```
git push origin
```

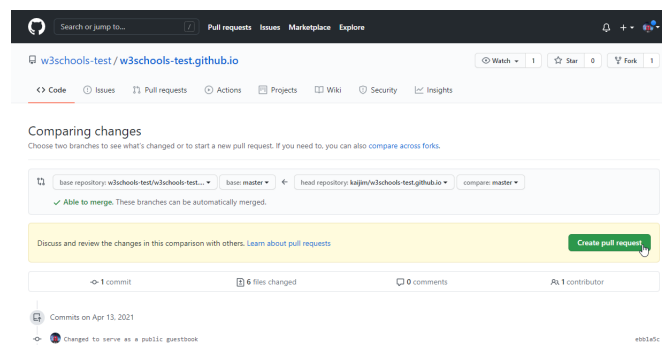


Figura 6: Ejemplo de pull request (w3schools.com)

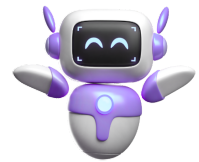
Queda en sus manos ver si te acepta el pull request o no.

De igual forma, habras notado que el proceso es muy similar a cuando trabajas con tus propios repositorios remotos. La unica diferencia es que tus propios pull request se aceptan automaticamente, ya que **tu eres el dueño del repositorio!**

Tip: Buenas practicas con Git y Github

Aqui te dejo una serie de buenas practicas para que tu flujo con Git y Github sea una buena experiencia:

1. **Commits atomicos:** Que cada commit que hagas tenga un proposito unico (correccion de error, nueva funcionalidad, eliminacion de algo, etc.)
2. **Mensajes de commit cortos y claros:** "Tipo: Descripcion breve"
(ej: Add: Pantalla de login)
3. **Branches dirigidas:** Usalas para implementar nuevas funcionalidades y evita trabajar directamente en main. (main debe ser tu rama que sabes que siempre funciona)
4. **Ignora archivos innecesarios:** Usa `.gitignore` para excluir archivos innecesarios en tu repositorio remoto.
5. **Manten tus datos a salvo:** NUNCA HAGAS PUSH DE CONTRASEÑAS O DATOS SENSIBLES!
6. **Sincroniza frecuentemente con Github:** Siempre manten sincronizada tu version local con la remota haciendo `pull` y `push`.



3. Variables y tipos de datos

En esta sección se verá el concepto de variables y de por qué son tan importantes a la hora de programar, junto a ello también se verán los tipos básicos (**primitivos**) que posee Python para almacenar datos.

3.1. Variables, el corazón del código

En casi cualquier programa, será necesario manipular o almacenar datos para crear un flujo/lógica deseada. De no ser así, el programa solo sería algo estático que se ejecutaría exactamente igual una y otra vez (algo así como solo imprimir el mismo texto una y otra vez, etc).

Aquí es donde entran las **variables**, que son básicamente espacios en la memoria para almacenar datos. Puedes imaginártelas como cajitas con nombre que tienen un valor dentro, el cual puede ser un nombre, número, etc.

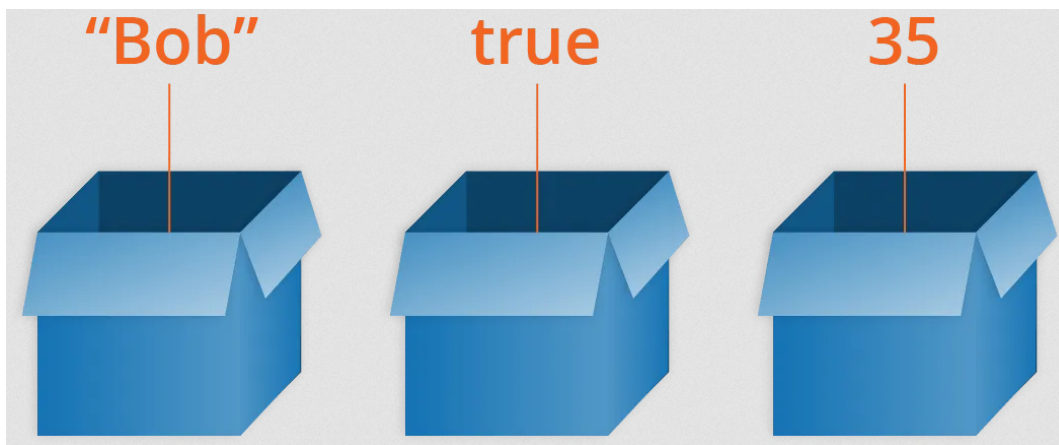
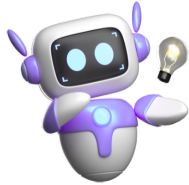


Figura 7: Analogía de variables como cajitas con valores

Dato freak: Concepto de variables

Una variable no es mas que un nombre entendible por el humano para referirse a una dirección en la memoria del computador. Es decir, cuando tengo una variable que se llama **a** y vale **10** por ejemplo, realmente **"a"** es solo un camuflaje para una dirección en memoria que se ve algo así: **FF0x1A** y tiene almacenado el valor **10**. Entonces, por eso pueden existir choques de nombres cuando corre nuestro programa, ya que el interpretador de Python no sabría a que dirección de memoria ir y conseguir el valor. (para resolver esto existe el **scope** que es un concepto que se vera mas adelante)



¿Como declaro variables?

Para guardar algo como variable, basta con ponerle un nombre siguiendo las reglas mencionadas en la sección anterior y asignándole un valor.

```
#Estas son variables
a = 10
b = "Hola"

print(a) #Imprimira 10
print(b) #Imprimira 'Hola'
```

Aquí si nos damos cuenta, ya tenemos el poder de hacer que el programa sea mas dinámico, ya que las variables pueden ir cambiando su valor a lo largo del código.

Variables que cambian su valor mientras corre el programa

```
#Esta variable inicialmente vale 10
a = 10

print(a) #Imprimira 10

a = 5 #Cambiamos su valor mientras corre el programa

print(a) #Imprimira 5
```

3.2. Tipos básicos de datos

Los valores almacenados por la variables también tienen sus propios tipos, ya que por ejemplo: **no es lo mismo un número que una palabra** no interactúan igual entre si. Por lo tanto, Python debe tener un indicio de qué rayos está guardando en cada variable o que se puede hacer con el dato en cuestión.

Para eso existen los **tipos de datos**, cada uno tiene sus propias funciones e interacciones específicas.

En otros lenguajes sería necesario especificar el tipo de cada variable antes de empezar a correr el programa, sin embargo una de las ventajas de que Python sea **interpretado** es que él se encarga de darles un tipo mientras el programa se ejecuta (automático) así que no debes preocuparte de eso (aun)

3.2.1. Int

Son números enteros (sin decimales), es compatible con todas las operaciones aritméticas y de comparación entre números.

Números enteros: Int

```
#Estos son un numero entero para python
a = 10
b = 5

type(a) #Int
type(b) #Int

#Se pueden operar entre si
suma = a+b # 15
resta = a-b # 5
multiplicacion = a*b # 50
division = a/b # 2
```

3.2.2. Float

Son números decimales, poseen todas las características de los Int.

Decimales: Float

```
#Estos son numeros decimales para Python
a = 10.5
b = 5.156

type(a) #Float
type(b) #Float

#Tambien se pueden redondear

a = round(a) # ahora vale 11
b = round(b, 2) # ahora vale 5.16
```

La funcion **round(numero, n)** devuelve el numero redondeado a **n** decimales.

3.2.3. Bool

Es un **booleano**, es decir solo puede ser Verdadero o Falso (**True/False**). Sirve por lo general para indicar si un programa está listo, si algo es cierto o no, etc.

Verdadero/Falso: Bool

```
#Estos son los booleanos para Python
a = True
b = False

type(a) #Bool
type(b) #Bool

#Cualquier cosa != 0 sera True

a = bool(5) # True
a = bool(0) # False
a = bool("Hola") # True
```

3.2.4. String (str)

Son palabras, tienen que ir entre 2 " y poseen varios métodos útiles para trabajarlos de manera mas sencilla. (estos métodos los veremos en detalle mas adelante)

Palabras: String

```
#Estas son las palabras para Python
a = "Hola"
b = 'Chao'

#Notar que se pueden usar tanto comillas simples ( ' ') como dobles ( " " ),
#pero no se deben mezclar entre si

type(a) #String
type(b) #String

#Tambien se pueden operar (mas adelante se vera en detalle)

ambas = a + b
print(ambas) # HolaChao

a = a*5
print(a) # HolaHolaHolaHolaHola
```

3.2.5. Resumen

Resumen de tipos de datos

```
#Resumen tipos de variables en Python

a = 5 #Int (número entero)

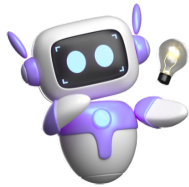
a = 5.6 #Float (número decimal)

a = True #Bool (verdadero/falso)

a = "Hola Mundo" #String (palabra)
```

3.3. ¿Como recibir datos del usuario?

Ahora, imaginemos que queremos crear un programa en donde el usuario le asigne valor a nuestras variables. Es eso posible?



Como recibir input del usuario

La funcion **input()** nos permite pedirle al usuario que introduzca un valor en la consola y el código se quedara detenido hasta que dicho valor sea introducido. Posterior a eso, el valor introducido por el usuario sera devuelto como un **string** por la funcion **input()**.

```
#Asi se reciben y guardan datos del usuario

a = input()

type(a) #String

print(a) #Imprimira lo que sea que puso el usuario

#-----
#Notar que se debe guardar en algún lado o usarla de inmediato
#De lo contrario la función será llamada en vano

#No hace nada por si sola, devuelve lo escrito por el usuario,
# pero no hay donde guardarlo :(
input()

#Imprimira en pantalla lo escrito por el usuario
 #(usar de inmediato)
print(input())
```

Recordar siempre que **input()** devuelve lo que sea que haya escrito el usuario en una linea como un **string**, por lo que quizás haya que hacerle alguna **conversión** si es que se quiere trabajar de otra forma con el dato (numero, bool, etc).

Tratar correctamente inputs

```
#Asegurarse de recibir bien inputs

#Asumamos que queremos hacer un programa que reciba 2 números (a y b)
#Luego queremos que los sume e imprima el resultado en pantalla

#Notar que también se puede pedir input junto a una frase :D

a = input("Introduzca primer numero: ")
b = input("Introduzca segundo numero: ")
```

```
# Asumamos que el usuario puso a = 12 y b = 5

'''
Esto de aquí abajo estará malo, ya que en vez de sumar 2 NÚMEROS (a y b)
tratará de sumar 2 STRINGS.
Lo cual resultara en que el programa imprima "125"
en vez de 17 (resultado deseado)
'''

print(a + b) # 125 :(

'''
La manera correcta sería convertir ambas variables a INTS y
luego sumarlas
'''

a = int(a)
b = int(b)
print(a + b) # 17 :)
```

3.4. Conversión entre tipos de datos

A veces, será necesario que cambies el tipo de un dato a otro (string $\Rightarrow$ int, int $\Rightarrow$ float, etc.) para ello existe algo llamado **casting** de variables, que no es mas que cambiarles el tipo (como Python los trata) para poder trabajar con ellos en algún otro contexto.

Para ello, Python es bien simple, simplemente se debe colocar el nuevo tipo de dato que se desea y entre paréntesis la variable que se quiere **castear**.

Conversión de tipos (casting)

```
#Conversion de tipos

#Asumamos que a y b son 2 variables random

b = 10 # b inicialmente vale 10 (int)

# Como serían las conversiones?

a = int(b) # 10

a = float(b) # 10.0

a = bool(b) # True

a = str(b) # "10"

# -----
# Python no siempre será capaz de castear

c = "Hola"
```

```
d = int(c) #ERROR: ¿Cómo diablos transformo 'Hola' a un número?
```

NO siempre se podrán convertir tipos de datos entre sí, por ejemplo: Python no sabe como convertir el string "Hola" a un número. Por lo cual si intentamos algo como `int("Hola")` el programa fallara épicamente.

Dato freak: Conocer el tipo de dato que tiene guardada una variable

Para conocer que tipo de dato tiene una variable guardada, existe el comando `type()` en Python.

```
a = 10

tipo = type(a)
print(tipo) #Imprimira 'int'

a = "Hola"

tipo = type(a)
print(tipo) #Imprimira 'str'
```

Tip: Imprimir varias cosas a la vez

Con el comando `print()` puedes imprimir varias cosas a la vez usando `,` para separarlas.

```
print("Hola") #Hola

#-----

a = 15

# "Tengo 15 botellas"
print("Tengo", a, "botellas")

#-----

a = "Sushi?"
b = "Pollo?"

print("Quieres", a) #"Quieres Sushi?"
print("Quieres", b) #"Quieres Pollo?"
```

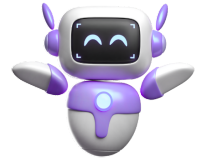


Tip: Formatear Strings complejos

Si necesitas incluir información de tus variables en tus strings puedes hacerlo fácilmente usando **f-strings**. Estos te dejan incluir cualquier cosa de Python (variables, funciones, etc) de manera sencilla en tu string.

```
nombre = "Lionel Messi"
edad = 38
texto = f"Hola, soy {nombre} y tengo {edad} años"

#Hola, soy Lionel Messi y tengo 38 años
print(texto)
```



3.5. Ejercicios de práctica

Reto: Variables básicas

Practica creando variables simples: números enteros, decimales, booleanos y cadenas de texto (strings).

Reto: Conversiones

Pide al usuario un número como texto, conviértelo a entero y muestra el doble.

Ejemplo

Input

5

Output

10

Reto: Entrada y tipos

Pide la edad al usuario y muestra qué tipo de dato tiene antes y después de convertirlo a entero.

Ejemplo

Input

21

Output

Antes: str, Después: int

3.6. Soluciones a los ejercicios de práctica

Solucion: Variables básicas

La idea de este reto es que te acostumbres a crear variables con distintos tipos básicos en Python.

```
a = 10 #int
b = "Hola que tal" #str

pepito = 5.5 #float
lector_bacan = True #bool
cupo_libre_en_el_ofg = False #bool
```

Solucion: Conversiones

Convertimos la entrada de texto a entero y calculamos el doble.

```
numero = input()
numero = int(numero) * 2
print(numero)
```

Solucion: Entrada y tipos

Mostramos el tipo de dato antes y después de la conversión usando la función `type()`.

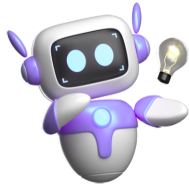
```
numero = input()
print("Antes:", type(numero))

numero = int(numero)
print("Despues:", type(numero))
```

4. Mas tipos de datos

4.1. Listas

Una lista es básicamente un **"contenedor de cosas"**. En ella puedes guardar varias cosas (incluyendo variables) juntas. Python las guarda de manera **contigua** en memoria.



Declaración de una lista

```
mi_lista = [item_1, item_2, item_3...]
```

#Ejemplo

```
a = 10
```

```
mi_lista = [1, 2, "Hola", "Chao", a]
```

4.1.1. Acceder a elementos dentro de una lista

Para acceder a un elemento de una lista, basta con poner **"lista[indice]"**. Donde indice es la posición dentro de la lista del elemento a acceder.

A diferencia de las personas, los computadores comienzan a contar desde el numero 0 en vez de el 1. Por lo tanto los indices de una lista de **N** elementos irán desde **0** $\Rightarrow$ **N-1**.

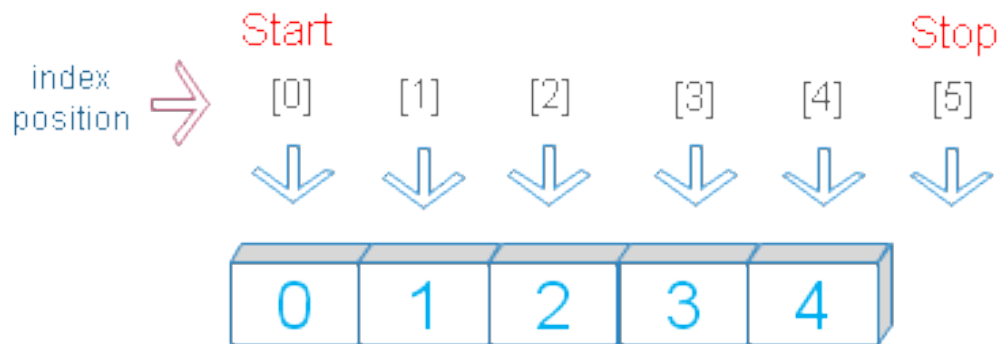


Figura 8: Indices de una lista de 5 elementos

¡Cuidado!

Cuando intentes acceder a un elemento de una lista, procura no pasarte de **largo** con el indice. Ya que el programa arrojara error (*index out of range*) y se detendrá por completo.



Otra cosa a destacar, es que el índice al cual queremos acceder también puede ser negativo, esto último Python lo interpretara como que estamos contando de **"atrás para adelante"** y podremos acceder mas fácilmente a los últimos elementos de la lista. Por lo tanto, puede ahorrarnos bastante código y ajeteo saber esta técnica.

Indices negativos dentro de una lista?

```
#Lista de 6 elementos
#Indices van [0-5]

mi_lista = [1, 2, 3, 4, 5, 6]

print(mi_lista[0]) #1
print(mi_lista[3]) #4
print(mi_lista[-1]) #6
print(mi_lista[-4]) #3

#ESTE CODIGO DA ERROR (Indice se pasa)
print(mi_lista[6]) #???
```

4.1.2. Métodos útiles de las listas

Las listas son un poderoso objeto en Python, ya que son **mutables** (pueden cambiar después de haber sido creadas) y tienen métodos para búsqueda, inserción, etc.

- **append(elemento):** Agrega un elemento al final de la lista
- **clear():** Vacía la lista entera
- **pop(idx):** Remueve y devuelve el elemento en posición 'idx' de la lista (si se deja en blanco por defecto es la parte trasera de la lista (**n-1**)).
- **count(elemento):** Devuelve cuantas ocurrencias de 'elemento' hay en la lista
- **copy():** Crea una copia de la lista
- **reverse():** Invierte el orden de la lista completa
- **insert(pos, elemento):** Inserta 'elemento' en la posición 'pos' de la lista, corriendo al resto de elementos hacia el lado.
- **sort():** Ordena la lista de menor a mayor (*veremos esto mas adelante igual*)

Métodos mas comunes usados en listas

```
#Aca hay varias listas

lista_a = [1, 2, 3, 4]
lista_b = ["Hola", 4, "Chao"]
lista_c = ["Pizza", 3.14]

#-----

# Modificar elementos
lista_a[0] = "Pera"
print(lista_a) # ["Pera", 2, 3, 4]
```

```

lista_a[2] = 3.14
print(lista_a) # ["Pera", 2, 3.14, 4]

#-----

# append -> Agregar elemento al final
lista_c.append("Manzana")
print(lista_c) # ["Pizza", 3.14, "Manzana"]

#-----

# clear -> Vaciar lista
lista_a.clear()
print(lista_a) # []

#-----

# pop -> quitar un elemento
lista_b.pop()
print(lista_b) # ["Hola", 4]

lista_c.pop(1)
print(lista_c) # ["Pizza", "Manzana"]

#-----

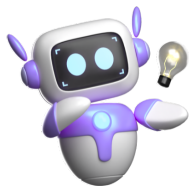
#count -> cuantas ocurrencias?
lista_a = [1, 1, 2, 3, 3, 3, 3, 3]

print(lista_a.count(1)) # 2
print(lista_a.count(2)) # 1
print(lista_a.count(3)) # 5

```

4.1.3. Sliceo de listas

Una de las ventajas que tienen las listas de Python es que pueden copiarse muy fácilmente sub-segmentos de estas mismas, usando una técnica llamada **slicing**.



Slicing de listas

```

lista_a = [1, 2, 3, 4, 5, 6]

#-----

lista_b = lista_a[1 : 3] # [2, 3]
lista_c = lista_a[3 : 5] # [4, 5]

lista_d = lista_a[2 : ] # [3, 4, 5, 6]
lista_e = lista_a[ : 3] # [1, 2, 3]

```

Básicamente, cuando decimos **lista[a : b]** le decimos al programa que queremos una parte de la lista inicial que va desde el índice **a** y termina justo antes del índice **b**.

También se puede agregar un tercer **argumento** (opcional) que indica el **step** que va a ir tomando, es decir va a ir avanzando de **step** en **step** índices hasta llegar a justo antes del índice **b**.

Sliceo con 3 argumentos

```
lista_a = [1, 2, 3, 4, 5, 6]

#-----

lista_b = lista_a[1 : 3] # [2, 3]
lista_c = lista_a[1 : 5 : 2] # [2, 4]

lista_d = lista_a[ : : -1] # [6, 5, 4, 3, 2, 1]
lista_e = lista_a[ : 3 : -1] # [6, 5]
```

No es obligatorio aprenderse el método de **slicear** pero es altamente recomendado, ya que ahorra bastante código si se quiere solo una parte de la lista original, siguiendo algún patrón.

r

4.2. Un poco mas acerca de los Strings

Ahora que ya tenemos algo mas de conocimiento acerca de las listas, podemos definir a un String de manera completa:

Un String es simplemente una lista de caracteres (1 sola letra/símbolo/etc.). Por lo tanto se pueden usar la mayoría de métodos que tienen las listas y mas! (slicing, acceso, etc)

4.2.1. Métodos de Strings

También los string tienen métodos extra que son útiles a la hora de

- **String[idx]:** Se accede al carácter en la posición "idx" del string.
- **.find(texto):** Devuelve la primera posición en la que se encuentre "texto" dentro del string. Devuelve **-1** si no esta presente.
- **.upper():** Devuelve el string completo transformado a solo letras **mayúsculas**.
- **.lower():** Devuelve el string completo transformado a solo letras **minúsculas**.
- **.replace(texto_1, texto_2):** Devuelve un string con todas las ocurrencias de "texto_1" reemplazadas por "texto_2" en el string.
- **.strip():** Devuelve una copia del string sin cualquier espacio en blanco innecesario al inicio y final del string.
- **.split(separador):** Devuelve una listas. Su función es cortar el string en pequeños strings, los cortes serán en toda posición en que "separador" este presente (esto elimina al separador también del string).
- Agregar todos los **métodos de listas** que conocemos...

Métodos de Strings

```

#Usaremos esta como palabra base
palabra = "Hola Mundo"

#-----
#Podemos indexar como si fuera una lista
print(palabra[2]) # l

#-----
#.find(texto): Buscar primer indice que contenga "texto" en el string
indice = palabra.find("undo")
print(indice) # 6

#-----
#.upper(): crea una copia del string en MAYUSCULAS
a = palabra.upper()
print(a) # HOLA MUNDO

#-----
#.lower(): lo mismo pero en minusculas
a = palabra.lower()
print(a) # hola mundo

#-----
#.replace(texto_a, texto_b): reemplaza todos los "texto_a" por "texto_b"
nueva_palabra = palabra.replace("o", "e")
print(nueva_palabra) # Hela Munde

#-----
#.strip(): Crea una copia del string sin espacios en blanco innecesarios
a = "    arroz chaufan    " # notar los espacios extra

b = a.strip()
print(b) #arroz chaufan

#-----
#.split(separador): Corta el string en mini strings
palabra = "Quiero comer sushi"
lista = palabra.split(" ") #la cortara en todos los espacios blancos " "

print(lista) # ["Quiero", "comer", "sushi"]
print(lista[0]) # Quiero

```

Eso sí, a diferencia de las listas, los String son **inmutables**. Lo que quiere decir es que cuando cambiamos una variable que sea un string por otro valor (a pesar de que sea otro string). Python lo que hace realmente por debajo es crear un **String nuevo** y sobrescribir el que ya existe.

¡Cuidado!

Debido a que son objetos **inmutables**, los String no pueden ser modificados parcialmente (al menos no de manera directa). Es decir, no se puede hacer algo como esto:

```
palabra = "Quiero pollo"

palabra[2] = "a" # ERROR

#No se puede modificar 'in place' un string
#(reemplazarle uno de sus elementos)

#-----
#Lo que si se puede hacer, es crear una
#copia del string ya modificada.

#Para casos como estos es util el slicing (ahorra codigo)

palabra_2 = palabra[ : 2] + "a" + palabra[3: ]
print(palabra_2) # Quaero pollo
```



4.3. Mutabilidad (dato freak)

Ya hemos mencionado bastante esta palabra a lo largo de esta guía, sin embargo, no le hemos dado una definición formal aún.

Esta definición la iremos ampliando a medida que avance la guía. Pero por ahora hay que abstraernos un poco a la idea de que **todo en Python es un objeto que posee una dirección en memoria y un valor asociado**. (esto se ve mas adelante en **programación orientada a objetos**)

Asumamos que tenemos 2 variables, **a** y **b**.

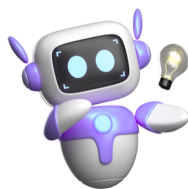
```
a = 10
b = [1, 2, 3]
```

Ambas tienen su **identificador**(a / b) y sus valores (10 y b respectivamente). Ahora, asumamos que queremos modificar sus valores.

```
a = 5
b[0] = 10

print(a) # 5
print(b) # [10, 2, 3]
```

Resulta que para **a** que era un int (**immutable**) lo que pasa realmente por debajo es que se le asigna una nueva dirección en memoria

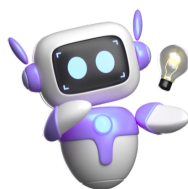


Que es la mutabilidad?

Un tipo de dato **Immutable** es todo aquel que **no** puede ser cambiado una vez creado, es decir: si se quiere cambiar, Python por detrás debe crear un nuevo valor con el resultado deseado (numeros, Strings, etc).
Mientras que un dato **Mutable** puede ser modificado **"on the go"** como las listas, diccionarios, etc.

4.4. Tuplas

Una tupla no es mas que una lista **immutable** con largo y valores fijos. Sirve por lo general para guardar un conjunto de datos que no cambia su formato durante todo el programa o para prevenir hacerlo.



Declaración de una tupla

```
mi_tupla = (1, 2, "Hola", etc...)
```

4.4.1. Métodos de tuplas

Las tuplas son algo mas limitadas que las listas, pero al tener elementos fijos, tienen algo mas de agilidad para ser accedidas durante la ejecución del programa. (mas eficientes)

- **.count(elemento):** cuantas ocurrencias de 'elemento' hay en la tupla?
- **.index(elemento):** busca el indice en el que se encuentra elemento, si no existe, devuelve -1.

Métodos de tuplas

```

tupla = (1, 1, 1, "Hola", 2, "Zorrito")

#-----
#.index(elemento): busca el primer indice donde este el elemento

print(tupla.index(1)) # 0
print(tupla.index("Hola")) # 3
print(tupla.index(3)) # -1 (no encontrado)

#-----
#.count(elemento): cuantas ocurrencias?

print(tupla.count(1)) # 3
print(tupla.count("Piña")) # 0

#-----

'''
Recordar siempre que son inmutables, por lo cual,
si tratamos de modificar un elemento ya asignado.
El programa tirara error.
'''

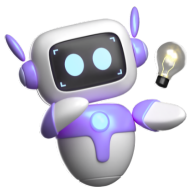
print(tupla[0]) #1 (ningun problema)

tupla[0] = 3 # ERROR: Tupla es immutable

```

4.5. Sets

Un set es básicamente una lista que posee elementos **inmutables** y **únicos**. Es decir, previene que haya elementos repetidos dentro de él.



Declaración de un set

```
mi_set = {1, 2, 3, "Si", "No", etc...}
```

4.5.1. Métodos de sets

Su traducción directa al español sería **conjunto** y tiene métodos muy similares a los de la definición matemática de **conjunto**.

Métodos básicos

- **.add(cosa):** agrega "cosa" al set. Si es que "cosa" ya existe dentro del set, no hace nada. (nada de elementos repetidos)

- **.discard(cosa):** Quita "cosa" del set
- **.clear():** Vacía todo el set
- **.copy():** Retorna una copia del set

Métodos básicos de sets

```

set_a = {1, 2, 3, 4, 5}
set_b = {3, 4, 5, 6, 7, 8}

#Metodos basicos

#-----
#.add(cosa): Agrega "cosa" al set

set_a.add(8)
print(set_a) # {1, 2, 3, 4, 5, 8}

set_a.add(3) # No hace nada (ya existe)
print(set_a) # {1, 2, 3, 4, 5, 8}

#-----
#.discard(cosa) Le quita "cosa" al set

set_a.discard(2)
print(set_a) # {1, 3, 4, 5, 8}

#No pasa nada si es que el elemento no existe :)
set_a.discard(10)

#-----
#.clear(): vacia todo el set

set_a.clear()
print(set_a) # {}

#-----
#.copy(): retorna una copia del set
set_a = set_b.copy()

print(set_a) # {3, 4, 5, 6, 7, 8}
print(set_b) # {3, 4, 5, 6, 7, 8}

```

Métodos de conjuntos

- **.difference(otro_set):** Retorna un set con solo elementos que no sean comunes entre ambos sets.
- **.difference_update(otro_set):** Funcionamiento similar a **difference()**, pero en vez de retornar una copia, modifica el set actual y le quita los elementos compartidos con **.otro_set**.
- **.intersection(otro_set):** Retorna una copia del set y que solo contiene los elementos compartidos entre ambos.

- **.intersection_update(otro_set):** Modifica el set actual para que solo tenga elementos compartidos por ambos.
- **.union(otro_set):** Retorna un set que contiene la **union** de ambos conjuntos (es como simplemente sumarlos entre si)
- **.update(otro_set):** modifica el set actual para que tenga **todos** los elementos de ambos sets. (mismo funcionamiento que en listas)

Métodos de conjuntos

```
set_a = {1, 2, 3, 4, 5}
set_b = {3, 4, 5, 6, 7, 8}

#Metodos de conjuntos

#-----
#.difference(otro_set): Retorna un set con elementos no comunes entre ambos
set_c = set_a.difference(set_b)
print(set_c) # {1, 2, 6, 7, 8}

#.difference_update(otro_set): Lo mismo que .difference pero afecta directamente al set
# (no crea copias)

set_d = {1, 2, 3, 4, 5}
set_d.difference_update(set_b)
print(set_d) # {1, 2, 6, 7, 8}

#-----
#.intersection(otro_set): Retorna un set con solo elementos comunes
set_c = set_a.intersection(set_b)
print(set_c) # {3, 4, 5}

#.intersection_update(otro_set): Lo mismo que .intersection
# pero afecta directamente al set (no crea copia)

set_d = {1, 2, 3, 4, 5}
set_d.intersection_update(set_b)
print(set_d) # {3, 4, 5}

#-----
#.union(otro_set)
set_c = set_a.union(set_b)
print(set_c) # {1, 2, 3, 4, 5, 6, 7, 8}

#.update(otro_set): lo mismo que .union pero no crea copia
set_a.update(set_b)
print(set_a) # {1, 2, 3, 4, 5, 6, 7, 8}
```

Métodos de comparación

- **.isdisjoint(otro_set):** Retorna True si es que no hay elementos compartidos entre sets.

- `.issuperset(otro_set)`: Retorna True si es que el set contiene **todos** los elementos del otro set.
- `.issubset(otro_set)`: Retorna True si es que el otro set, contiene **todos** los elementos de el set actual.

Métodos de comparación de sets

```
set_a = {1, 2, 3, 4, 5}
set_b = {4, 5, 6, 7, 8}
set_c = {1, 2, 3}

#Metodos de comparacion de sets

#-----
#.isdisjoint(otro_set): son sets totalmente diferentes? (no hay elementos comunes)

set_a.isdisjoint(set_b) #False (tienen elementos comunes)
set_b.isdisjoint(set_c) #True (nada en comun)

#-----
#.issuperset(otro_set): otro_set esta contenido dentro mio?

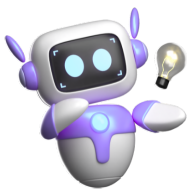
set_b.issuperset(set_a) #False (no todo lo de "a" esta en "b")
set_a.issuperset(set_c) #True (Todo lo de "c" esta tambien en "a")
set_c.issuperset(set_a) #False (no funciona al reves)

#-----
#.issubset(otro_set): Este set esta contenido en otro set? (todos son elementos comunes)

set_a.issubset(set_b) #False
set_a.issubset(set_c) #False (es super set de "c")
set_c.issubset(set_a) #True (todo "c" esta tambien en "a")
```

4.6. Diccionarios

Los diccionarios son como una lista **desordenada** que en vez de indexarse con **números**, lo hace mediante un sistema de **llave-valor**. Es decir, puedes usar una palabra o número para acceder a los contenidos del diccionario



Declaración de un diccionario

```
mi_diccionario = {"Nombre": "Steve", "Apellido": "Gonzales", etc..}
```

Como su nombre lo indica, funcionan como un **diccionario de la vida real**. Sí hay alguna **llave** (palabra, número, etc.) registrada en él. Esta tendrá asociada un **valor** (cualquier tipo de dato, incluso otro diccionario!).

4.6.1. Acceder a los elementos de un diccionario

Restricciones de las llaves

Una llave puede ser cualquier tipo de dato **immutable** de Python (Int, String, Bool, Float, Tuplas, etc.).

Restricciones de los valores del diccionario

Un diccionario puede almacenar cualquier valor, este no tiene por qué ser **immutable** como las llaves.

¡Cuidado!

A pesar de que los diccionarios pueden usar cualquier llave. Si se intenta acceder con una llave que no está registrada en él (no posee un valor asociado), el código arrojará error. (**KeyError**)

```
mi_dict = {"Nombre": "Steve", "Apellido": "Arroz", "ID": 839}

print(mi_dict["Nombre"]) #Steve

print(mi_dict["Casa"]) #ERROR: llave no encontrada
```

Para resolver este problema existen 2 soluciones:

- Usar un **defaultdict** (No se vera en esta guía en detalle).
- Usar el método **.get()** de los diccionarios. (se vera mas abajo)



Dato freak: Como un diccionario sabe a qué dirección de memoria acceder? (hashing)

Seguro te estarás preguntando, como es que un diccionario puede usar algo como un simple **string**, **bool**, **int**, **etc.** y mapearlo a una dirección de memoria donde hay un valor.

Esto se logra mediante una técnica llamada **hashing**, que básicamente consiste en tener un **algoritmo de hash** que le asocia a cada llave una dirección en memoria mientras corre el programa. Es decir, cuando le ponemos una llave como **"hola"** el algoritmo va a hacer sus cosas locas y le devolverá al diccionario la dirección **0xffaa** por ejemplo.

Hay muchos de estos algoritmos, cada uno con sus pros y contras en cuanto a eficiencia, facilidad de implementar, etc. Pero no hay que preocuparse mucho de ellos, ya que por lo general todos los lenguajes de programación lo integran por debajo para que no tengamos que hacerlo a mano.

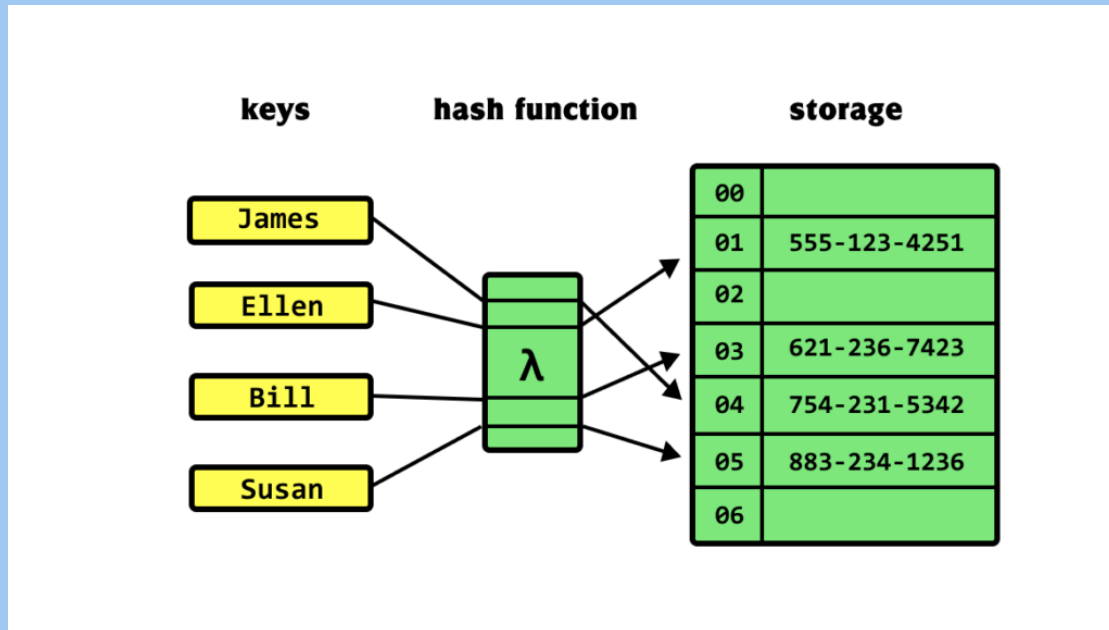


Figura 11: Representacion hashing (www.spiceworks.com)

4.6.2. Métodos de diccionarios

Los diccionarios tienen varios métodos al igual que las listas. Sin embargo, son ligeramente distintos en cuanto a su funcionamiento.

Métodos básicos

- **.get(llave, default):** Funciona como un **dict[llave]** pero en caso de que no exista la llave, devuelve "default" como respuesta en vez de crashear el programa con un error.
- **.pop(llave):** Elimina la llave y el valor asociado a ella del diccionario.
- **.clear():** Vacía el diccionario
- **.copy():** Retorna una copia del diccionario
- **.update(otro_dict):** Es como **sumar 2** diccionarios. Modifica el diccionario y le agrega todo lo que tenga el otro dict. Sin embargo **sobrescribirá cualquier llave repetida con el valor que este en el otro dict**.

Métodos básicos de diccionarios

```

mi_dict = {"Nombre": "Steve", "Apellido": "Gonzales", "Numero": 567932913}
mi_dict_2 = {1: "Kevin", 2: "Sofia", 3: "Trini"}

#-----
#Obtener un valor

mi_dict["Nombre"] # "Steve"

mi_dict["RUT"] #ERROR: llave no existe

#-----
#.get(llave, valor): Es una manera segura de tratar de obtener un valor
# exista o no

mi_dict.get("Nombre", "nada") # Steve

mi_dict.get("RUT", "nada") # nada (no existe la llave RUT)

#-----
#.pop(llave): elimina la llave y el valor asociado del diccionario

mi_dict.pop("Apellido")
print(mi_dict) # {"Nombre": "Steve", "Numero": 567932913}

#-----
#.clear(): vacia el diccionario

mi_dict.clear()
print(mi_dict) # {}

#-----
#.copy(): retorna una copia del diccionario

mi_dict = mi_dict_2.copy()
print(mi_dict) # {1: "Kevin", 2: "Sofia", 3: "Trini"}

#-----
#.update(otro_dict): Le agrega mas cosas al diccionario usando otro diccionario

otro_dict = {1: "Agustin", 4: "Cata"}
mi_dict.update(otro_dict)
print(mi_dict) # {1: "Agustin", 2: "Sofia", 3: "Trini", 4: "Cata"}

```

Métodos útiles (en especial para control de flujo)

- **.keys()**: Retorna una "lista" con **solo las llaves** del diccionario
- **.values()**: Retorna una "lista" con **solo los valores** del diccionario
- **.items()**: Retorna una "lista" de **tuplas** con todos los pares "llave-valor" del diccionario.

Metodos utiles de dicts

```
mi_dict = {1: "Kevin", 2: "Sofia", 3: "Trini"}

'''
Estos metodos devuelven objetos que directamente no son listas,
mas adelante veremos que por lo general no hay que hacerles modificaciones
ya que se ocupan en bucles normalmente.

Sin embargo, ten en cuenta que para estos ejemplos los transformaremos a listas para
poder imprimirlos en pantalla de manera correcta, nada mas.
'''

#-----
#.keys(): Devuelve una lista con solo las llaves

llaves = mi_dict.keys()
llaves = list(llaves)
print(llaves) # [1, 2, 3]

#-----
#.values(): Devuelve una lista con solo los valores

valores = mi_dict.values()
valores = list(valores)
print(valores) # ["Kevin", "Sofia", "Trini"]

#-----
#.items(): Devuelve una lista de tuplas con los pares llave-valor

pares = mi_dict.items()
pares = list(pares)
print(pares) # [(1, "Kevin"), (2, "Sofia"), (3, "Trini")]
```

4.7. Stacks y Colas

Estas son estructuras usadas con el fin de simplificar procesos de agregar/quitar objetos de algo.

- **Stacks:** Siguen un flujo **LIFO** (*last in first out*), es decir, primero debes sacar lo último que agregaste antes de poder sacar el resto de cosas (imagínatelo como una pila de platos, no puedes sacar el plato de abajo sin sacar los platos que pusiste al último arriba)
- **Colas:** Siguen un flujo **FIFO** (*first in first out*), es decir, lo que entra primero saldrá primero que todo el resto que coloques (imagínatelas como una fila del supermercado, siempre saldrá primero la persona que llego primero)

4.7.1. ¿Cómo hago una tupla o un stack en Python?

En Python no existe directamente "stack" o "queue" pero si existe una estructura que es capaz de imitar ambas en una sola, esta se llama **deque**. Es capaz solo de agregar y eliminar elementos en sus extremos (inicio y final).
0.8deque.pngFuncionamiento de una deque (geeksforgeeks)

Métodos básicos de deque

```

from collections import deque

#Existe la estructura deque
mi_stack = deque(["A", "B", "C"])
miCola = deque(["A", "B", "C"])

#Tambien se puede imitar comportamiento usando listas normales
mi_stack_alternativo = ["A", "B", "C"]

#Para insertar elementos de los extremos
mi_stack.append("D") #derecha
miCola.appendleft("D") #izquierda
mi_stack_alternativo.append("D")

#miCola = ["D", "A", "B", "C"]
#mi_stack = ["A", "B", "C", "D"]

#Extender usando otra lista
mi_stack.extend(["E", "F", "G"])
miCola.extendleft(["1", "2", "3"])

#miCola = ["3", "2", "1", "D", "A", "B", "C"]
#mi_stack = ["A", "B", "C", "D", "E", "F", "G"]

#Quitar cosas
mi_stack.pop()
miCola.popleft()

#miCola = ["2", "1", "D", "A", "B", "C"]
#mi_stack = ["A", "B", "C", "D", "E", "F"]

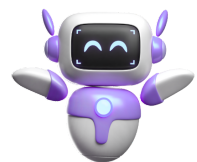
#Vaciarlas
miCola.clear()
mi_stack.clear()

#miCola = []
#mi_stack = []

```

Tip: Uso de listas simples para imitar comportamientos

Los comportamientos de un stack o cola se pueden imitar tambien usando listas simples, simplemente se deberia usar `.append()` y `.pop()` todo el tiempo.



4.8. Tabla comparativa estructuras de datos

Aquí hay un resumen de las principales estructuras de datos en Python y sus características.

Estructura	Lista	Tupla	Set	Dict
¿Mutable?	Sí	No	Sí	Sí (llaves fijas)
¿Ordenado?	Sí	Sí	No	Sí (desde Python 3.7)
¿Permite duplicados?	Sí	Sí	No	Solo en valores
Uso común	Varios datos	Datos fijos	Valores únicos	Pares llave-valor
Sintaxis	[1, 2, 3]	(1, 2)	{1, 2, 3}	{"llave": "valor"}

Cuadro 1: Comparación de estructuras de datos en Python

4.9. Named Tuples

Son como las tuplas, pero puedes acceder a sus elementos con una palabra, como si de un **objeto** se tratase. Requieren de un **módulo** llamado **collections**, veremos mas adelante que son los módulos, sin embargo, por ahora simplemente hay que agregar esta linea al comienzo del código:

```
from collections import namedtuple
```

Declaración named tuples

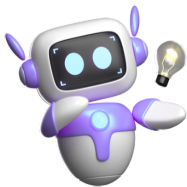
```
#Linea importante para poder usar las namedtuple
from collections import namedtuple

# Declaracion de una namedtuple
Nombre_namedtuple = namedtuple(nombre_namedtuple, lista_de_atributos)

#-----
#Ejemplo

Persona = namedtuple('Persona', ['Nombre', 'Edad', 'RUT'])

# Crear una persona nueva
alvin = Persona('Alvin', 18, '119.982.872-K')
```



Acceso a elementos de una namedtuple

```
# Namedtuple creada por nosotros
Persona = namedtuple('Persona', ['nombre', 'edad', 'RUT'])

# Crear 2 personas nuevas
alvin = Persona('Alvin', 18, '119.982.872-K')
diego = Persona('Diego', 21, '829.762.625-9')

#-----
#Acceso por indice
print(alvin[0]) # 'Alvin' (nombre)
print(alvin[1]) # 18 (edad)

#Acceso por nombre del atributo
print(diego.nombre) # 'Diego'
print(alvin.RUT) # 829.762.625-9
```

4.10. Ejercicios de práctica

Reto: Listas básicas

Crea una lista con varios números y muestra el cuarto elemento.

Reto: Manipulación de strings

Recibe una palabra y muestra la palabra en mayúsculas.

Ejemplo 1

Input

hola

Output

HOLA

Reto: Tuplas simples

Crea una tupla con tres elementos y muestra su longitud.

Reto: Sets y eliminación de duplicados

Recibe una lista de números separados por espacios y muestra la cantidad de elementos únicos.

Ejemplo 1**Input**

1 2 2 3 3 3 4

Output

4

Reto: Diccionarios básicos

Crea un diccionario con 3 pares clave-valor y muestra el valor asociado a una clave dada.

Ejemplo 1**Input**

clave2

Output

valor2

Reto: Named Tuples

Crea una namedtuple llamada 'Persona' con campos nombre y edad, luego crea una instancia y muestra el nombre.

4.11. Soluciones a los ejercicios de práctica

Solucion: Listas básicas

Creamos una lista y accedemos al elemento en la posición 3 (cuarto elemento).

```
lista = [2, 3, 4, 6, 1, 2, 4, 0, 10, -5]
print(lista[3])
```

Solucion: Manipulación de strings

Usamos la funcion `.upper()`.

```
palabra = input()
print(palabra.upper())
```

Solucion: Tuplas simples

Creamos la tupla y usamos `len()` para obtener la cantidad de elementos.

```
mi_tupla = ("a", 3, "siuu")
print(len(mi_tupla))
```

Solucion: Sets y eliminación de duplicados

Convertimos la lista a set para eliminar duplicados y luego contamos sus elementos.

```
mi_lista = input().split()
mi_set = set(mi_lista)

print(len(mi_set))
```

Solucion: Diccionarios básicos

Creamos un diccionario cualquier y accedemos a la llave que nos indique el usuario.

```
mi_dict = {"Hola": "Mundo", "Harry": "Styles", "Vendemos bolsos": "pero no hay reembolsos"}

clave = input()
print(mi_dict[clave])
```

Solucion: Named Tuples

Definimos la namedtuple, creamos una instancia y accedemos al campo nombre.

```
from collections import namedtuple

Persona = namedtuple("Persona", ["nombre", "edad", "altura"])

pepito = Persona("pepito", 20, 1.8)
print(pepito.nombre)
```

5. Funciones: ¡No repitas código!

5.1. ¿Qué es una función?

Una función en Python se puede ver de manera similar a una **maquina mágica** que recibe 0 o mas valores y **retorna** (o **no**) un resultado.

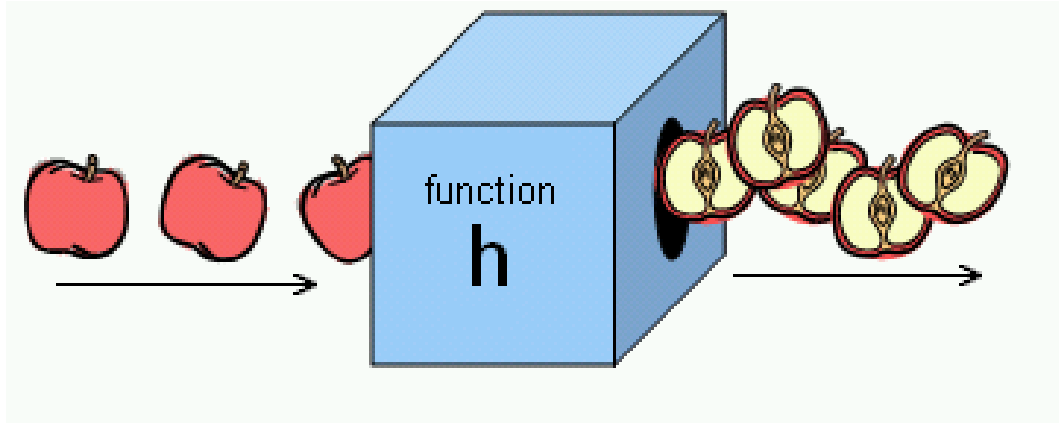


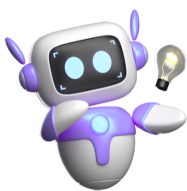
Figura 12: Representación de una función

Una función no es mas que un "bloque" de código al cual le asignamos un nombre, recibe 0 o mas valores y nos retorna un valor cuando es ejecutada. Su objetivo principal es poder rehusar y encapsular código.

Dato freak: DRY

"Don't repeat yourself" es uno de los principios fundamentales a la hora de desarrollar software: Se basa en la idea de no repetir código que ya escribimos en alguna parte del programa, facilitando la mantención y ampliación a largo plazo de este mismo.

5.2. Sintaxis y uso



Sintaxis de una funcion

```
def nombre_funcion(parametro1, parametro2, parametro3, ...):  
    #Cuerpo de la funcion  
  
    return valor_retorno
```

La cantidad de parámetros queda a tu criterio a la hora de programar, por lo general dependerá mucho de que es lo que quieres que haga tu función.

El cuerpo de la función es el bloque de código que quieres reutilizar (aquí es donde haces cosas con tus variables, cálculos, etc).

Ejemplo: Uso de una función

```
def AreaRectangulo(base, altura):  
    return base*altura  
  
#Para usar una función solo debes llamarla usando ()  
#El valor de retorno quedará asignado a la variable  
  
#Se le pueden pasar los argumentos por posición  
#(orden = orden de nombres en la función)  
a = AreaRectangulo(10, 20)  
  
#Tambien puedes pasarle los argumentos por palabra clave  
#(no importa el orden)  
b = AreaRectangulo(base=30, altura=10)  
  
c = AreaRectangulo(altura=500, base=2)
```

5.3. Argumentos y parámetros

Los **parámetros** de una función son los nombres que se le dan a los valores recibidos, mientras que los **argumentos** son los valores recibidos por esta misma.

- **Argumento:** Valor real recibido a la función
- **Parámetro:** Nombre que le da la función al valor (piénsalo como un **"alias"** para el valor que le pasas realmente a la función (**argumento**). El cual existe solo dentro de la función)

Para entender como Python interpreta estos valores, debemos recordar la definición de 4.1.3 **Mutabilidad...** Esta definición es algo confusa al inicio, pero es crucial, ya que Python por debajo los pasa de manera distinta a la función según su mutabilidad:

- **Argumentos inmutables:** A la función se le pasa una **copia** del valor real, es decir lo que pase dentro de la función se queda dentro de la función y el parámetro desde afuera no es afectado.
- **Argumentos mutables:** A diferencia de los inmutables, Python los pasa directamente **por referencia**, es decir: Cualquier modificación que se la haga al parámetro dentro de la función se vera reflejada en el exterior.

Ejemplo: argumentos por copia

```
def funcion_magica(parametro):  
    #Hago algo con el parametro dentro de la funcion  
    parametro = "Conejo"  
    return parametro  
  
#Valores inmutables (numeros y Strings)  
valor = 10  
resultado = funcion_magica(valor)  
  
# --> Conejo / La funcion funciona normalmente  
print(resultado)  
  
# ---> 10 / No fue afectado  
print(valor)
```

Ejemplo: argumentos por referencia

```
def funcion_magica(parametro):  
    #Hago algo con el parametro dentro de la funcion  
    parametro.append("Conejo")  
  
    #Tratamos de cambiarla totalmente  
    parametro = "Conejo"  
    return parametro  
  
#Valores mutables (listas, diccionarios, etc)  
valor = [1, 2, 3]  
resultado = funcion_magica(valor)  
  
# --> "Conejo" / La funcion funciona normalmente  
print(resultado)  
  
# ---> [1, 2, 3, "Conejo"]  
# Fue afectada la lista y ahora tiene "Conejo"  
print(valor)  
  
#Notar que a pesar de haber podido ser modificada (agregar algo)  
#El tratar de cambiarla a otro tipo de dato (línea 6) no tuvo efecto  
#Esto indica que el parametro es solo una referencia modificable  
#del real y no su direccion en memoria (dato freak)
```

5.4. Valores de retorno

¡Cuidado!

El comando **return** solo puede ser usado dentro de funciones y simplemente lo que hace es:

1. Detener la función
2. Retornar lo que le pongamos a su derecha (**return** cosa)



Recalco este punto, ya que la función se detiene por completo y no se llega al código que este debajo y muchas veces puede pasar algo como esto:

Ejemplo: mal uso del return

```
#Funcion que retorna e imprime la suma de a y b
def sumar(a, b):
    suma = a + b
    return suma

#Esto no se ejecutara :(
print(f'La suma es {suma}!')
```

5.4.1. ¿Y si no quiero devolver nada?

También puede suceder que no queramos devolver nada y simplemente imprimir algo en pantalla, agregar cosas a una lista, etc.

Python se encarga de esto usando la "nada" o mas bien **None**.

Este es un nombre reservado de Python y literalmente se usa para describir "la nada", no es nada!

Por lo tanto, una función sin un **return** es equivalente a una función que retorna **None**, es decir: nada!

Ejemplo de retorno None

```
#Funcion que solo imprime y no retorna nada
def saludar(nombre):
    print(f"Holaa {nombre}")

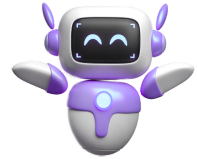
#Si la ejecuto...
saludar("Miguel") # ---> Imprime 'Hola Miguel' en pantalla

#Si la intento asignar a una variable..
a = saludar("Paula") # ---> No retorna nada, por lo que a sera None

#Veamos que es a
print(a) # ---> None
```

Tip: Retorno de una función

Se puede colocar manualmente que una función retorne **None**, sin embargo. Si no le ponemos `return`, Python sólo se encargará de no retornar nada, por lo cual no es necesario.



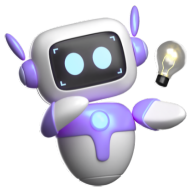
5.5. *Args y **Kwargs (Avanzado)

Que pasa si no sabemos realmente cuantos argumentos nos van a dar? pueden ser 1, 2, 5 o 10! quien sabe? Para esto, Python tiene implementados los **operadores de desempaquetamiento**: (`*` y `**`), que indica a la función que va a recibir una cantidad arbitraria de valores.

- ***args**: Recibirá argumentos **posicionales** de 1 en 1 (la cantidad que sea), para luego transformarlos a una tupla a la que puedes acceder dentro de la función. (`args[0]`, `args[1]`, etc..)
- ****kwargs**: Recibirá argumentos por **keyword** (cantidad que sea) y los transformará a un diccionario al cual podemos acceder dentro de la función. (`kwargs['nombre']`, `kwargs['numero']`, etc..)

Uso de *args y **kwargs

```
def funcion_con_parametros_fijos(valor_1, valor_2, etc..):  
    #cosas de funciones  
  
#-----  
  
def funcion_con_parametros_variables(*args):  
    #cosas de funciones  
    args[0] #acceder al primer argumento entregado  
    print(args[1])  
    #etc...  
  
#-----  
  
def funcion_con_parametros_keyword_variables(**kwargs):  
    #cosas de funciones  
    kwargs["numero"] #acceder al argumento 'numero'  
    print(args['nombre'])  
    #etc...  
  
#-----  
  
def un_poco_de_todo(valor_1, *args, **kwargs):  
    #tu imaginacion es el limite!
```



Ejemplo: Sumar cantidad variable de números (*args)

```
def sumador(*args):
    suma = 0
    for i in args:
        suma += i
    return suma

print(sumador(1, 2)) #--> 3 (1+2)
print(sumador(1, 5, 7)) #--> 13 (1+5+7)
print(sumador(10, 3, 1, 3, 6)) #--> 23 (10+3+1+3+6)
```

Ejemplo: Uso de **kwargs

```
def funcion_diccionario(**kwargs):
    for llave, valor in kwargs.items():
        print(f"{llave} --> {valor}")

#-----

funcion_diccionario(nombre='Miguel', numero='9278356281212')

print('')
print('segunda llamada')
print('')

funcion_diccionario(a=10, b=5, hola='chao')

'''
-----
#OUTPUT:

nombre --> Miguel
numero --> 9278356281212

segunda llamada

a --> 10
b --> 5
hola --> chao
'''
```

Es un estándar usar ***args** para los parámetros variables y ****kwargs** para los parámetros con keywords variables. Sin embargo, como al fin y al cabo son parámetros, podemos darle el nombre que queramos. (eso si es buena practica mantener el estándar)

Ejemplo: nombres propios para parámetros variables

```
def mi_funcion_loca(*cosas, **otras_cosas):  
    #sumar todas las cosas  
  
    suma = 0  
    for i in cosas:  
        suma += i  
    print("Suma:", suma)  
  
    #Imprimir a los parametros keyword  
    for llave, valor in otras_cosas.items():  
        print(f"{llave} --> {valor}")  
  
#-----  
  
mi_funcion_loca(1, 2, 3, 4, nombre='Martina', telefono='Iphone')  
  
'''  
OUTPUT:  
  
Suma: 10  
nombre --> Martina  
telefono --> Iphone  
'''
```

¡Cuidado!**CUIDADO A LA HORA DE MEZCLAR!**

Si bien, puedes hacerlo es importante que siempre sigas el orden:

```
def funcion(valores_fijos, *args, **kwargs)
```

Cualquier otra mezcla rara resultara en un error, ya que Python no sabrá como interpretar los argumentos que le estés pasando (fijo o variable?, *args o **kwargs?, etc..)

```
#Ordenes correctos

def mi_funcion(a, b, *args, **kwargs):

def mi_funcion(a, *args):

def mi_funcion(a, **kwargs):

def mi_funcion(*args, **kwargs):

#-----
#Ordenes incorrectos (ERROR)

def mi_funcion(*args, a, **kwargs):

def mi_funcion(a, **kwargs, *args):

def mi_funcion(**kwargs, a, *args):

def mi_funcion(a, *args, b)
```

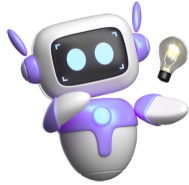


5.6. Scope de una función (avanzado)

Hasta ahora hemos hablado de que a veces existen "mini universos" en donde solo existen algunas variables, por ejemplo:

- Fuera de una función no existen los parámetros (nombres que inventamos adentro)
- Fuera de un 'for' no existe la variable iteradora (i)
- Toda variable que creamos dentro de una función tampoco esta disponible para usarse afuera

Pero que pasa realmente?



Scope

Cuando creamos una variable, esta esta disponible solo desde adentro de la región en donde la creamos. A esta **”region”**, en donde están disponible nuestra variables, se llaman **Scopes**. Existen 2 tipos:

- **Scope global:** Disponible para todo el programa, esta declarada fuera de cualquier función o bucle (no tiene indentacion).
- **Scope local:** Se refiere a el Scope local de alguna función, clase, bucle, etc. Cualquier variable **declarada** dentro, solo estará disponible dentro de ese scope.

Ejemplo: Scope global

```
a = 10 #esta variable es visible para todo el codigo

def mi_funcion():
    print(a)

mi_funcion() #10 (esta usando la variable global 'a')

a = 15

mi_funcion() #15 (esta usando la variable global 'a')
```

Ejemplo: Scopes locales

```
a = 10 #esta variable es visible para todo el codigo

def mi_funcion():
    #Esta variable pertenece al scope de la funcion 'mi_funcion'
    a = 15
    b = 20

    #Cualquier modificacion no se vera reflejada en la global
    #Por lo tanto, cualquier cambio en la global tampoco se vera
    #reflejado aca dentro

    print(a)

#-----

mi_funcion() # 15 (local)

print(a) # 10 (la global no sufrio cambios (nunca la pescaron) )
a = 30

mi_funcion() #15 (local)
# (esta usando la variable local 'a' que inicializo con '15')

#Todo lo declarado dentro del scope local no existe en scopes externos

print(b) #ERROR --> Scope externo: ¿Que es b?
```

Los Scope funcionan de **afuera hacia adentro**, es decir: "Toda variable declarada en un scope que este mas afuera, también estará disponible para los Scope de mas adentro". Por eso las variables **globales** están disponibles para todo el programa! Son el Scope superior a todos.

Ejemplo: Múltiples scopes

```
variable_global = "Hola"

def mi_funcion():
    variable_local_1 = 10

    #aqui solo puedo usar a global y local_1
    print(variable_global)
    print(variable_local_1)

    for i in range(10):
        print(variable_global)
        print(variable_local_1)
        print(i)

        #aqui dentro puedo usar a cualquiera

    print(i) #ERROR --> No existe i aqui

#aqui solo existe variable_global

print(variable_global) #--> Hola
print(variable_local_1) #ERROR --> No existe
print(i) #ERROR --> No existe
```

Ahora, te estarás preguntando: "Si tengo 2 variables que se llaman igual, pero en distinto scope. ¿Cual usara Python?"

La respuesta es: "La del Scope mas cercano".

Ejemplo: Mismo nombre

```
a = 10

def mi_funcion():
    a = 15
    print("A desde mi funcion --> " + a)

print("A desde afuera --> " + a) #--> 10
mi_funcion() #--> 15

#Notar que creamos una variable nueva llamada 'a' que pertenece al scope de la funcion
#No afectamos a la 'a' global

print(a) #--> 10
```

¡Cuidado!

A veces pasa que las funciones se confunden entre si estas **asignándole un valor nuevo** a una variable que ya existe en un **Scope externo** o si estas **declarando una nueva** dentro de su Scope. Si notas que tus funciones no se están comportando como deberían o que sus variables no están cambiando de valor, probablemente tengas conflictos entre scopes que te están impidiendo acceder a la variable que quieres.



Afortunadamente, Python tiene 2 keywords que nos ayudan en casos como estos:

- **global:** Se usa si quieres especificarle a una función que use una **variable global** que ya declaraste anteriormente.
- **nonlocal:** Se usa si quieres especificarle a una función que use una **variable de un scope externo** al suyo. (se usa si es que tienes **funciones declaradas dentro de funciones**, pero esa parte se va del enfoque de esta guía. (aquí tienes un ejemplo igualmente)

Ejemplo: uso de global

```
a = 10

def mi_funcion():
    #Usar el 'a' global (no crear una nueva variable)
    global a

    a = 15
    print("Cambiando la variable global...")

print(a) #--> 10
mi_funcion() #--> Cambiando la variable global...
print(a) #--> 15
```

Ejemplo: uso de nonlocal

```
a = 1 #esta variable es global

def mi_funcion():
    a = 10 #esta variable pertenece al scope de mi_funcion

    def mi_funcion_2():
        #Quiero usar el 'a' de un scope externo al mio
        nonlocal a
        print("a desde adentro del todo:", a)

    mi_funcion_2()

mi_funcion()
print(a, "(global)") #No tocamos a la global!

#-----
#Output:

# a desde adentro del todo: 10
# 1 (global)
```

¡Cuidado!

Cuando Python se sale de un scope (una función/bucle termina) **TODO** lo declarado dentro del Scope es eliminado por completo del programa. Por eso podemos reutilizar los nombres nuevamente mas adelante en el programa.



5.7. Documentación de funciones (avanzado)

5.7.1. Type Hinting

Quizas te habrás dado cuenta que a veces es difícil saber que valores pasarle a una función (ints, floats, strings?) ya que solo conocemos el nombre de los parámetros recibidos. También pasa que no sabemos si una función retorna o no algo, y si es que retorna algo, que es? (String, int, float, bool?)

Type Hinting

Para que la simpleza de Python al declarar variables (no especificar de que tipo son) no termine en un caos a largo plazo, existe la **buena practica** del **Type Hinting** (agregada desde Python 3.5 en adelante), que consiste en darle una pista al programador acerca de los tipos de datos que deben ser tanto los **argumentos** como el **retorno** de una función.

Sintaxis type hint

```
#Sintaxis Type hinting

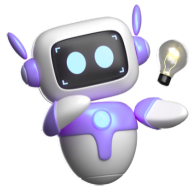
#Variables
nombre_variable: tipo

#Funciones (retorno)
def nombre_funcion() -> tipo_retorno:
    #blah bla (codigo de funcion)

#-----
#Ejemplos

a: int = 10
frutas: list = ["pera", "manzana", "platano"]

def contar_letras(palabra: str) -> int:
    return len(palabra)
```



Ejemplo: Una función con type hinting

```
def saludar(nombre: str) -> None:  
    print("Hola ", nombre, "!")
```

La función anterior especifica que debe recibir un **string** y que no devuelve nada (**None**).

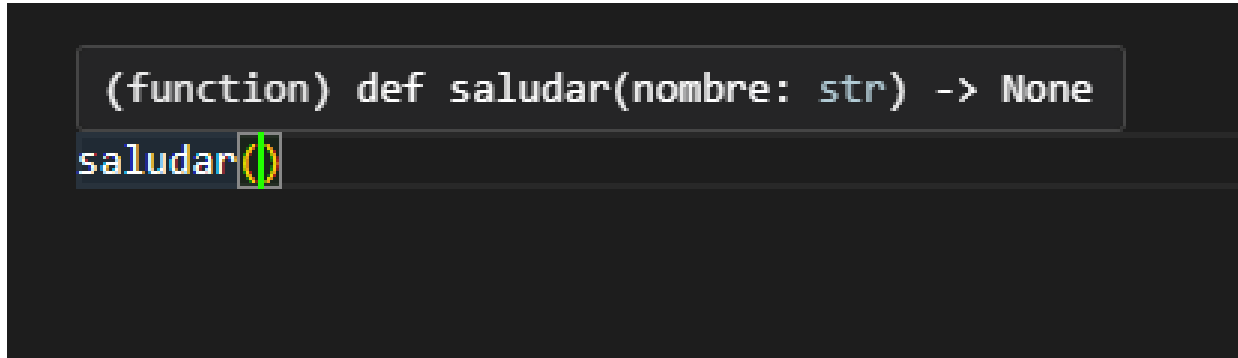


Figura 13: Ejemplo de como se vería la función desde afuera (visual studio code)

Tip:

Si bien, no es obligatorio usar **type hinting**, es una muy buena práctica para mantener tu código ordenado y legible.



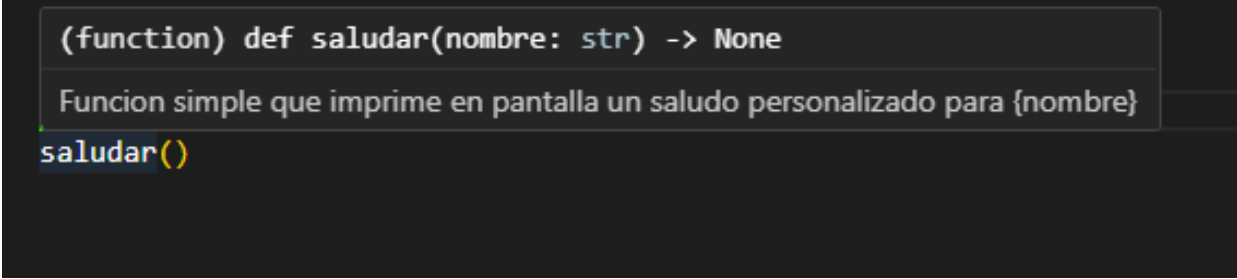
5.7.2. Docstrings

También, puedes agregarle texto a la función que sirve como **guía** (sale en el preview cuando vas a usar la función en algún editor de código). Para esto, simplemente agrega un comentario multilinea justo debajo del nombre de tu función.

Ejemplo: Función documentada con Docstring

```
def saludar(nombre: str) -> None:
    """
    Funcion simple que imprime en pantalla un saludo personalizado
    para {nombre}
    """

    print("Hola ", nombre, "!")
```



```
(function) def saludar(nombre: str) -> None
Funcion simple que imprime en pantalla un saludo personalizado para {nombre}
saludar()
```

Figura 14: Ejemplo de como se ve su preview en Visual Studio

Tip: Dale formato a tu documentación

Los docstring admiten formato **Markdown!**

Por lo cual puedes dejar aun mas ordenado y legible tu documentación acerca de tu función :)

Ejemplo: función con docstring en markdown

```
def saludar(nombre: str) -> None:
    """
    ## Funcionalidad
    Funcion simple que imprime en pantalla un saludo personalizado

    ## Argumentos
    **nombre (str)**: Nombre de quien quieres saludar

    ## Retorno
    Nada (None)
    """
    print("Hola ", nombre, "!")
```

```
17
18 (function) def saludar(nombre: str) -> None
19
20 Funcionalidad
21 Funcion simple que imprime en pantalla un saludo personalizado
22
23 Argumentos
24 nombre (str): Nombre de quien quieres saludar
25
26 Retorno
27 Nada (None)
28 saludar()
```

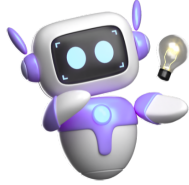
Figura 15: Ejemplo de como se ve el docstring con lenguaje markdown



5.8. Funciones lambda (avanzado)

Imaginemos que queremos simplificar una funcionalidad a una función para ahorrarnos líneas ilegibles de código. Pero que sucede si queremos usarla muy pocas veces (a veces incluso 1 sola vez) y no queremos que después quede usando memoria y un nombre en nuestro código?

Para ello, existen las **funciones anónimas** o mejor conocidas como: **funciones lambda**. Estas no tienen un nombre y por lo general se usan para cosas simples / temporales que requieran ser encapsuladas en una función.



Sintaxis funciones lambda

```
#Sintaxis de una funcion anonima (Lambda)  
  
lambda lista_argumentos: expresion_evaluable  
  
#-----  
#Ejemplo: funcion para sumar 2 numeros (a, b)  
  
lambda a, b: a+b
```

Ejemplo: uso de lambda

```
sumar = lambda a,b: a+b  
  
print(sumar(1, 2)) #--> 3  
print(sumar(5, 10)) #--> 15  
print(sumar(7, 28)) #--> 35
```

Ejemplo: uso de lambda con múltiples resultados

```
sumar = lambda a,b: a+b

'''
-----
#Este codigo es equivalente a decir:

    def sumar(a, b):
        return a+b

-----
'''

elevar_a_2 = lambda x: x**2

'''
-----
Este codigo es equivalente a decir:

    def elevar_a_2(x):
        return x**2

-----
'''

print(elevar_a_2(sumar(2, 3))) #--> 25 (5^2)

print(elevar_a_2(sumar(1, 7))) #--> 64 (8^2)
```

Lo curioso es que **lambda** devuelve una **función**, por lo cual puede ser asignada a una variable y luego ser usada. (esto rompe totalmente el propósito de lambda, pero ten en cuenta que puede simplificar su uso dentro de otras funciones / bucles, por ejemplo).

Ejemplo: Asignar lambda a una variable

```
# Esta funcion sumara todos los primeros n numeros elevados al cuadrado
#      1^2 + 2^2 + 3^2 + 4^2 ... n^2

def elevar_y_sumar(n):
    suma = 0
    for i in range(1, n+1):
        x = (i**2)
        suma += x

    return suma
```

Notar que sin asignarlo a una variable, si es que queremos cambiar la operación (elevar a 2 $\Rightarrow$ otra cosa) si no recordamos que parte modificar, podemos dejar un desastre. Arreglemoslo un poco para futuras modificaciones

```
# Esta funcion sumara todos los primeros n numeros elevados al cuadrado
#      1^2 + 2^2 + 3^2 + 4^2 ... n^2

def elevar_y_sumar(n):
    funcion = lambda x: x**2
    suma = 0
    for i in range(1, n+1):
        suma += funcion(i)

    return suma
```

5.9. Decoradores (@) (avanzado)

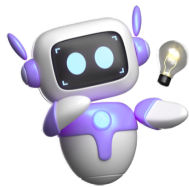
Los decoradores son básicamente **modificadores** para nuestras funciones, agregándoles / modificando su comportamiento.

Esencialmente son **funciones que reciben funciones y retornan funciones**, suena algo confuso pero hechemosle una mirada.

Sintaxis decoradores

```
def nombre_decorador(funcion_a_decorar):  
    def wrapper(*args, **kwargs):  
        #Hacer cosas  
  
        resultado = funcion_a_decorar(*args, **kwargs)  
  
        #Hacer cosas  
  
        return resultado  
    return wrapper  
  
#-----  
#Aplicarlo a alguna funcion  
  
@nombre_decorador  
def funcion_a_decorar():  
    #Funcionalidad normal de la funcion  
    #blah blah
```

- **nombre_decorador:** El nombre para identificar al decorador.
- **funcion_a_decorar:** Parámetro que representa la función que le vamos a pasar al decorador.
- **wrapper():** Funcion anidada dentro del decorador, sirve como **funcion auxiliar** que hará toda la mecánica de nuestro decorador, luego, debe ser el retorno de el decorador para ser usada por la funcion que queremos decorar (mejor explicado en los ejemplos mas adelante).
- ***args y **kwargs:** Representan los argumentos que le habrían pasado a la funcion original antes de ser decorada. Es importante especificarlos para que la funcion no pierda su funcion original.



Ejemplo: crear un print con separadores

```
def mi_decorador(funcion):
    def wrapper(*args, **kwargs):
        print("----- (linea superior)")

        resultado = funcion(*args, **kwargs)

        print("----- (linea inferior)")
        print(" ")

        return resultado
    return wrapper

@mi_decorador
def imprimir(cosa):
    print(cosa)

#-----
#Ejemplo de uso:

imprimir("Hola")
imprimir("Chao")

'''
-----
#Output:

----- (linea superior)
Hola
----- (linea inferior

----- (linea superior)
Chao
----- (linea inferior
'''
```

Notar que básicamente lo que estamos haciendo es **modificar** a la función **imprimir()** para que tenga decoración cuando imprima texto. Realmente lo que pasa por debajo cuando usamos un decorador es lo siguiente:

```
def imprimir(cosa):
    print(cosa)

imprimir = mi_decorador(imprimir)

#-----
#Equivalente a

imprimir = wrapper
```

De aquí en adelante, tu imaginación es el límite con los decoradores, solo ten en cuenta que afectará a todas las futuras llamadas de la función, sin excepciones.

Ejemplo: Uso de decoradores

```
def anti_errores(funcion_divisora):
    def wrapper(a, b):
        print(f"Quiero dividir ({a}/{b}), veamos si puedo...")

        if(b == 0):
            print("NO PUEDO DIVIDIR POR 0 :(")
            return

        print("Se pudo!")
        return funcion_divisora(a, b)

    return wrapper

@anti_errores
def dividir(a, b):
    return a/b

#-----
#Testeemos

print(dividir(625, 5)) #--> 125.0 (se pudo!)
print(dividir(1, 0)) #--> None (NO PUEDO DIVIDIR POR 0)

'''
-----
OUTPUT:

Quiero dividir (625/5), veamos si puedo...
Se pudo!
125.0
Quiero dividir (1/0), veamos si puedo...
NO PUEDO DIVIDIR POR 0 :(
None
'''
```

Ejemplo: Uso de decoradores anidados

```
def poner_lineas(func):
    def wrapper(*args, **kwargs):
        print("-----")
        func(*args, **kwargs)
        print("-----")
    return wrapper

def poner_estrellas(func):
    def wrapper(*args, **kwargs):
        print("*****")
        func(*args, **kwargs)
        print("*****")
    return wrapper

#Ponerle 2 decoradores anidados a mi funcion imprimir...

@poner_lineas
@poner_estrellas
def imprimir(cosa):
    print(cosa)

#-----

imprimir("Hola")

'''
-----
OUTPUT:

-----
*****
Hola
*****
-----
'''
```

El código anterior es equivalente a decir:

```
def imprimir(cosa):
    print cosa

imprimir = poner_lineas(poner_estrellas(imprimir))
```

5.10. Ejercicios de práctica

Reto: Definir funciones

Crea una función que reciba un nombre y salude con un mensaje personalizado.

Ejemplo 1

Input	Output
Mateo	Hola, Mateo!

Reto: Argumentos y parámetros

Escribe una función que calcule el área de un triángulo dados base y altura.

Ejemplo 1

Input	Output
5 3	7.5

Reto: Funciones y retorno

Haz una función que reciba una lista de números y devuelva la suma total.

Ejemplo 1

Input	Output
1 2 3 4 5	15

Reto: Ámbito de variables (scope)

Crea una función que modifique una variable global dentro de su cuerpo.

5.11. Soluciones a los ejercicios de práctica

Solucion: Definir funciones

Definimos la función con un parámetro y usamos print para el saludo.

```
def saludar(nombre):  
    print("Hola, " + nombre + "!")  
  
saludar(input())
```

Solucion: Argumentos y parámetros

Calculamos el área usando $\text{base} * \text{altura} / 2$ y devolvemos el resultado.

```
def area_triangulo(base, altura):  
    return (base*altura)/2  
  
#-----  
  
valores = input().split()  
area = area_triangulo(int(valores[0]), int(valores[1]))  
  
print(area)
```

Solucion: Funciones y retorno

Sumamos todos los números de la lista y retornamos la suma.

```
def suma_total(lista):  
    total = 0  
    for i in lista:  
        total += int(i) #recordar que input recibe strings  
    return total  
  
#-----  
  
lista = input().split()  
print(suma_total(lista))
```

Solucion: Ámbito de variables (scope)

Declaramos la variable global dentro de la función con la palabra clave **global** y la modificamos.

```
variable_global = "CR7"

def cambia_formas():
    global variable_global
    variable_global = "Messi"

#-----
print("Antes del cambio: " + variable_global)
cambia_formas()
print("Despues del cambio: " + variable_global)
```

6. Programación orientada a objetos (OOP)

6.1. ¿Que es la programación orientada a objetos?

La programación orientada a objetos es un **paradigma** de programación, el cual sirve para estructurar tu programa de modo que cada parte tenga su propio comportamiento de manera **abstraída** y simple.

Esto es muy útil para modelar cosas como juegos, bases de datos, simulaciones, etc. Por ejemplo: Imagina tienes un programa que simule una ciudad. La ciudad tiene autos, personas, calles, etc.

Con la programación orientada a objetos, podrías modelar las siguientes cosas como **objetos** (mas adelante veremos que son realmente):

- **Persona:** Tiene nombre, apellido, edad, dinero, etc.
- **Auto:** Tiene marca, modelo, patente, gasolina, etc.
- **Calle:** Tiene largo, ancho, cantidad de autos sobre ella, semáforos, etc.
Y así sucesivamente...

También cada objeto tiene comportamientos propios..

- **Persona:** Puede caminar, hablar, etc.
- **Auto:** Puede moverse, estacionarse, etc.
- **Calle:** Puede cambiar sus semáforos, romperse, etc.
Y así sucesivamente...

Si tratáramos de modelar todo esto y todas sus variantes con solo funciones y variables, nos volveríamos locos. Pero en cambio si encapsulamos cada cosa individual como un objeto con sus propias cualidades y funciones. Podemos lograr interacciones que se vean así:

```
persona.saludar()

auto.estacionarse()

calle.agregar_auto(auto)

print(persona.nombre)
```

Bonito, no?

6.2. Como aplicar OOP en Python

6.2.1. ¿Que es una clase?

Una clase en Python simplemente representa/modela alguna cosa (ya sea real o abstracta) que tiene un comportamiento propio.

Las **clases** representan una **plantilla de como debería modelarse** un objeto en específico, en ella se define todo el comportamiento y propiedades del objeto modelado.

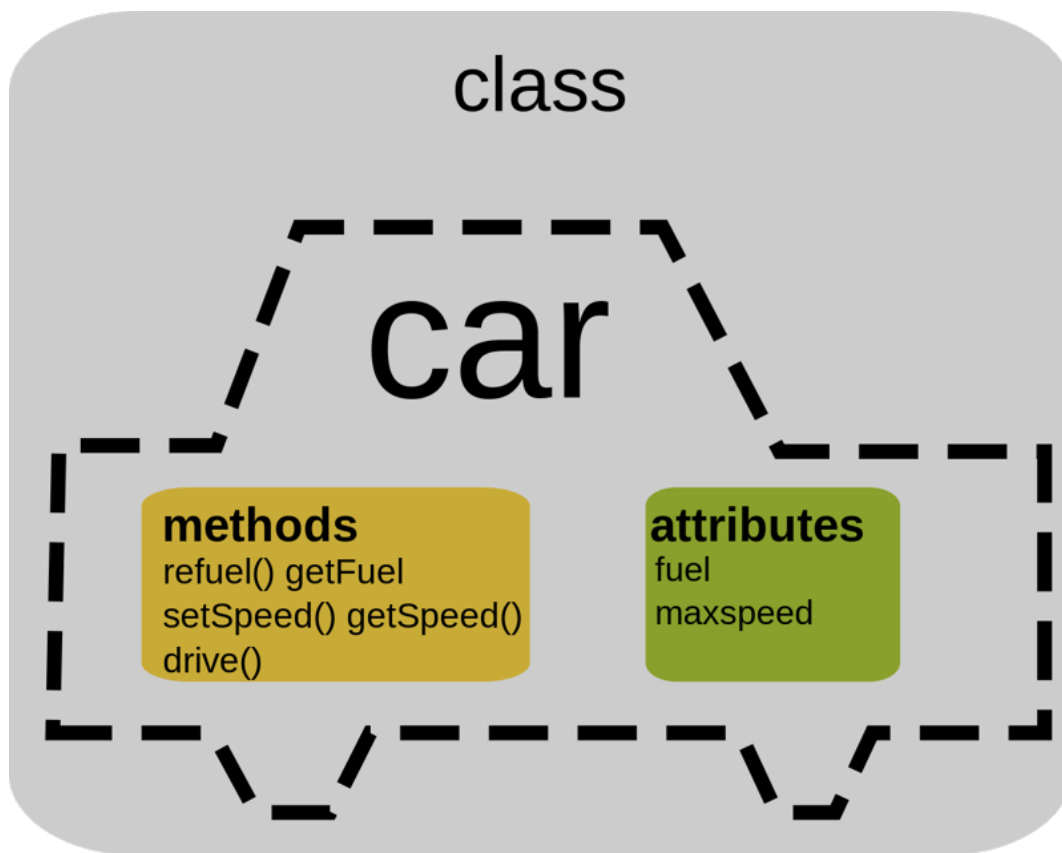


Figura 16: Representación de objetos en el paradigma OOP (logicmojo.com)

Todo lo que tenga que ver con **propiedades** del objeto, por ejemplo: nombre, edad, fuerza, velocidad, etc. Se le llaman **atributos**.

Mientras que todo lo que tenga que ver con **funciones** que puede hacer el objeto se le llaman **métodos**.

Como definir una clase en Python

```
class Nombre_clase:
    '''
    Aqui puedes colocar todos los atributos / variables
    que creas que necesita tu clase
    '''

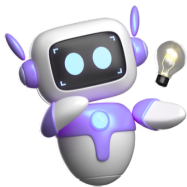
    #esta funcion es fundamental
    def __init__(self):
        #Esta sera llamada cada vez que instanciamos
        #Alguno de estos objetos (mas adelante veremos esto)

    '''
    Aqui ya puedes agregarle lo que tu quieras
    que tenga (metodos / atributos)
    '''

#-----
#Ejemplo:

class Persona:
    especie = "Homo sapiens"

    def __init__(self, nombre, edad):
        self.nombre = nombre
        self.edad = edad
```



6.2.2. Clases v/s instancias

Con el código del ejemplo anterior, solo tenemos la **plantilla** de como queremos que sean las 'Personas' en nuestro programa. Por ahora solo tienen **nombre y edad**. Cuando creamos un objeto a partir de una clase se llama **instanciar**. Que, valga la redundancia, significa crear una **instancia** de la clase en cuestión.

- **Clase:** Plantilla de como se modela cada objeto individual
- **Instancia:** Objetos individuales hechos a partir de la clase.

Cada **instancia** tiene una **identidad única**, a diferencia de solo el esquema, tiene:

- **Estado:** Cada objeto tiene atributos individuales.
- **Comportamiento:** Como se comportan sus métodos en base a sus atributos.

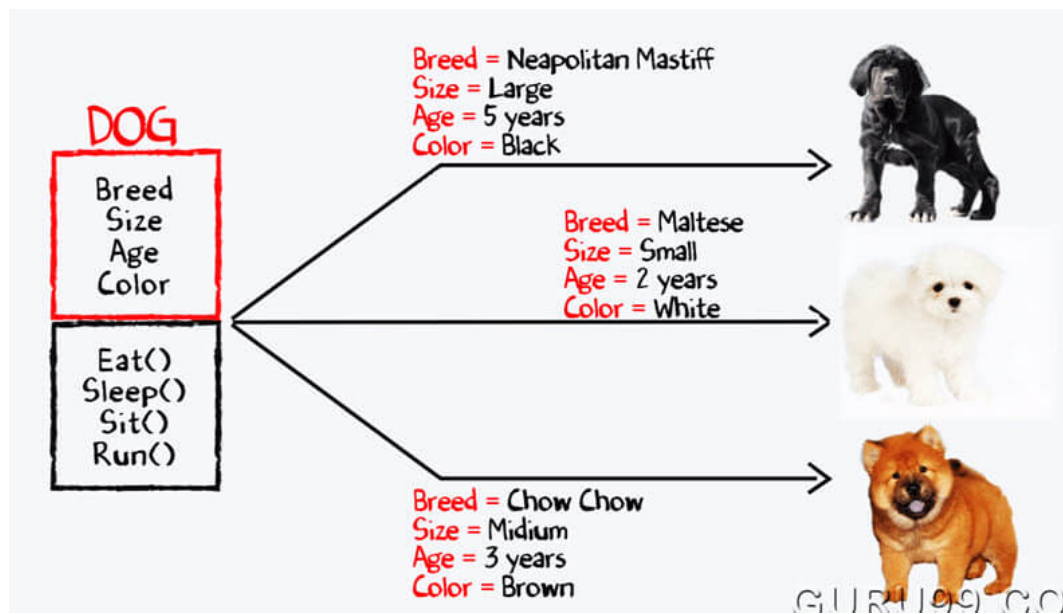


Figura 17: Ejemplo de Clases (Dog) v/s instancias (Lucas Bennett / guru99.com)

Crear instancias de una clase

```
#Clase: esquema de como deben ser nuestros objetos

class Persona:
    especie = "Homo Sapiens"

    def __init__(self, nombre, edad):
        self.nombre = nombre
        self.edad = edad

    def saludar(self):
        print("Hola, soy " + self.nombre + " y tengo " + str(self.edad))

#-----

#Instancias: Los objetos individuales

persona_1 = Persona("Billie", 22)
persona_2 = Persona("Lucas", 25)

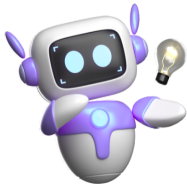
#Cada instancia tiene una identidad unica

persona_1.saludar() #--> Hola, soy Billie y tengo 22
persona_2.saludar() #--> Hola, soy Lucas y tengo 25

#-----

#Sin embargo, ambos siguen compartiendo los mismos atributos
#en comun con su clase

print(persona_1.especie) #--> Homo Sapiens
print(persona_2.especie) #--> Homo Sapiens
```



6.2.3. El parámetro "self" y la función __init__()

El parámetro **self** dentro de la clase, representa que nos referimos a un método o atributo que pertenece a la **instancia** y no a la clase. Es decir, todo lo que tenga **self**.algo se refiere al objeto individual. Mientras que todo lo demás es compartido por todos los objetos de la misma clase (en el ejemplo anterior se muestra esto).

Toda instancia, cuando llama a un método, lo hace pasándole **self** como **argumento**. Esto indica que se quiere usar sus atributos y métodos en particular, de lo contrario, el programa no sabría ni a que método específico llamar ni que atributos usar.

Por lo cual todo método de la clase debe contener como primer **parámetro** la palabra **self** para indicar que nos estamos refiriendo a un método que sera **único** para cada instancia. Si no lo hacemos, el programa arrojará error cuando tratemos de usarlos.

Una excepción a esto, son métodos que queremos que se comporten igual para todas las instancias, estos son los **métodos de clase y métodos estáticos**. Pero se salen del enfoque de esta guía, siéntete libre de investigarlos por tu cuenta.

El uso de la palabra self

```
#Clase bien hecha (con self)

class Persona:
    especie = "Homo Sapiens"

    def __init__(self, nombre, edad):
        self.nombre = nombre
        self.edad = edad

    def saludar(self):
        print("Hola, soy " + self.nombre + " y tengo " + str(self.edad))

#-----
#Clase mal hecha (sin self)

class Persona_mal_hecha:
    especie = "Homo Sapiens"

    def __init__(nombre, edad):
        nombre = nombre
        edad = edad

    def saludar():
        print("Hola, soy " + nombre + " y tengo " + str(edad))

#-----
#Pongamoslo a prueba

#Clase bien hecha (todo se comporta bien)

persona_1 = Persona("Bob", 30)
persona_1.saludar() #--> Hola, soy Bob y tengo 30
print(persona_1.nombre) #--> Bob

#Clase mal hecha (todo tirara error)

#ERROR --> __init__() no tiene como instanciar bien
#Recibe 3 argumentos (self, Thanos, 19) y solo aguanta 2 (nombre, edad)
persona_2 = Persona_mal_hecha("Thanos", 19)

persona_2.saludar() #ERROR --> No tengo nombre ni edad propia

print(persona_2.nombre) #ERROR --> Cual es mi nombre?
```

Al crear instancias, Python debe saber como **construirlas**, para ello existe el **método constructor** (`__init__()`). El cual es llamado una sola vez (cuando se crea el objeto), su misión es inicializar todos los atributos y métodos del objeto. Por lo tanto, debe recibir si o si de **primer** parámetro (**self**), pero la cantidad final de parámetros la pones tu (dependiendo como quieres modelar tu objeto)

Inicializacion de instancias

```
class Perro:
    def __init__(self, nombre, raza):
        self.nombre = nombre
        self.raza = raza
        print("woof woof, me llamo " + nombre)

class Galleta:
    def __init__(self, sabor):
        self.sabor = sabor
        print("hola, soy una galleta sabor " + sabor)

#-----
#__init__() se llama una sola vez (cuando se instancia el objeto)

perro_1 = Perro("Max", "Labrador") #--> woof woof, me llamo Max

perro_2 = Perro("Sam", "Pitbull") #--> woof woof, me llamo Sam

cookie = Galleta("Vainilla") #--> hola, soy una galleta sabor Vainilla

#-----
#Luego simplemente, podemos usar la clase como nos de la gana

print(perro_1.raza) #--> Labrador
print(cookie.sabor) #--> Vainilla
```

Que pasa por debajo cuando llamamos a los métodos de una clase?

```
class Perro:
    def __init__(self, nombre, raza):
        self.nombre = nombre
        self.raza = raza
        print("woof woof, me llamo " + nombre)

    def ladrar(self):
        print("Woof woof")
        print("Soy un " + self.raza + " y te estoy ladrando")

#-----

perro_1 = Perro("Max", "Labrador") #--> woof woof, me llamo Max

# Por debajo, Python hace esta llamada a __init__()
# perro_1.__init__(self, "Max", "Labrador")

perro_2 = Perro("Sam", "Pitbull") #--> woof woof, me llamo Sam

# Por debajo, Python hace esta llamada a __init__()
# perro_2.__init__(self, "Sam", "Pitbull")

#-----

perro_2.ladrar() #--> "Woof woof Soy un Pitbull y te estoy ladrando"

# Por debajo, Python hace esta llamada a ladrar()
# perro_2.ladrar(self)
```

6.3. ¿Como usar las instancias?

Como te habrás dado cuenta, para acceder y usar los métodos y atributos de alguna instancia, basta con solo colocar: **nombre_variable.cosa**, esa es la magia de OOP! (queda todo encapsulado y simple de usar).

Uso de instancias

```
class Perro:
    animal = "Perro"

    def __init__(self, nombre, raza):
        self.nombre = nombre
        self.raza = raza
        print("guau guau, me llamo " + nombre)

    def ladrar(self):
        print("Un " + self.raza + " y te esta ladrando")

    def comer(self, comida):
        print("A " + self.nombre + " le encanta comer " + comida)

#-----

#Instanciar objetos
p1 = Perro("Destructor de mundos", "Poodle")
p2 = Perro("Princesa", "Doberman")
p3 = Perro("Max", "Labrador")

#Acceder a sus metodos y atributos

p1.ladrar() #--> Un Poodle te esta ladrando
print(p3.nombre) #--> Max
print(p2.raza) #--> Doberman
p2.comer("Marraqueta") #--> A Princesa le encanta comer Marraqueta

#-----

#Notar que podemos hacer lo que queramos con los objetos

perros = [p1, p2, p3]

for i in perros:
    i.ladrar()

'''
Un Poodle te esta ladrando
Un Doberman te esta ladrando
Un Labrador te esta ladrando
'''
```

Usa tus objetos donde quieras!

```
class Libro:
    def __init__(self, titulo, autor, fecha):
        self.titulo = titulo
        self.autor = autor
        self.fecha = fecha
        self.precio = 0

    def dar_resumen(self):
        print(f"Titulo: {self.titulo}, Autor: {self.autor}, Publicacion: {self.fecha}")

    def establecerPrecio(self, precio):
        self.precio = precio

#-----
#Podemos usar nuestras clases donde queramos, incluso dentro de otras!

class Libreria:
    def __init__(self, nombre, jefe):
        self.nombre = nombre
        self.jefe = jefe
        self.dinero = 0
        self.libros = [] #Lista de 'Libros'

    def agregar_libro(self, libro, precio):
        libro.establecerPrecio(precio)
        self.libros.append(libro)

    def vender_libro(self, indice):
        #podemos usar los metodos de nuestra clase 'Libro'
        self.libros[indice].dar_resumen()
        self.dinero += self.libros[indice].precio
        self.libros.pop(indice)

#-----
#Ejemplo de uso:
#-----

#Crear 2 librerias independientes
libreria_1 = Libreria("Contra punto", "Luis")
libreria_2 = Libreria("Antartica", "Francisca")

#-----
#Agregarles libros a cada una

# Libro (Titulo, autor, fecha), precio
libreria_1.agregar_libro(Libro("Papelucho", "Marcela paz", 2025), 4500)

libro_1 = Libro("Percy Jackson", "Rick Riordan", 2005)
libreria_2.agregar_libro(libro_1, 9600)
```

```
#-----
#Vender y ver cuanto dinero tienen
libreria_1.vender_libro(0) #--> Titulo: Papelucho, Autor: Marcela paz, Publicacion: 2025
libreria_2.vender_libro(0) #--> Titulo: Percy Jackson, Autor: Rick Riordan, Publicacion: 2005

print(libreria_1.dinero) #--> 4500
print(libreria_2.dinero) #--> 9600
```

6.4. Dunder methods

Son métodos que están escritos de esta manera `--nombre_metodo--` y vienen definidos en Python. Estos métodos también son conocidos como los “**métodos mágicos**” y al sobrescribirlos podemos cambiar como se comportan las operaciones básicas de Python con nuestros objetos. (+, -, *, /, len, etc.).

Dunder Methods

Se llaman **Dunder** para simbolizar la palabra “Doble under”, por como están escritos (doble underscore).

Al sobrescribirlos, cambiamos el comportamiento base de Python con nuestros objetos, agregando funcionalidades bastante útiles en algunos casos. Algunos ejemplos son:

Operaciones de Python

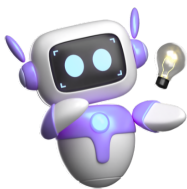
- `--init__(self, *args)`: Inicializar el objeto
- `--del__(self)`: Destructor del objeto (cuando se elimina de memoria)
- `--repr__(self)`: Representación “legible” del objeto (`print(objeto)`)
- `--sizeof__(self)`: Tamaño del objeto (`len(objeto)`)
- `--call__(self, *args)`: Comportamiento al llamar (`objeto()`)

Operaciones numéricas

- `--add__(self, otro)`: Comportamiento al sumar (`objeto + otro`)
- `--sub__(self, otro)`: Comportamiento al restar (`objeto - otro`)
- `--mul__(self, otro)`: Comportamiento al multiplicar (`objeto * otro`)
- `--truediv__(self, otro)`: Comportamiento al dividir (`objeto / otro`)
- `--floordiv__(self, otro)`: Comportamiento al usar división entera (`objeto // otro`)
- `--mod__(self, otro)`: Comportamiento al usar modulo (`objeto % otro`)
- `--pow__(self, otro)`: Comportamiento al elevar (`objetootro`)

Operaciones de comparación

- `--eq__(self, otro)`: Comportamiento igualdad (`objeto == otro`)
- `--lt__(self, otro)`: Comportamiento menor (`objeto < otro`)
- `--le__(self, otro)`: Comportamiento menor/igual (`objeto ≤ otro`)
- `--gt__(self, otro)`: Comportamiento mayor (`objeto > otro`)
- `--ge__(self, otro)`: Comportamiento mayor/igual (`objeto ≥ otro`)



Ejemplo: Operaciones con objetos 'Persona'

```
class Persona:
    especie = "Homo Sapiens"

    def __init__(self, nombre, edad):
        self.nombre = nombre
        self.edad = edad

    def __lt__(self, otra_persona):
        return self.edad < otra_persona.edad

a = Persona("Diego", 21)
b = Persona("Amelia", 18)

print(a < b) #--> (23 < 18) = False
```

Ejemplo: Creando nuestros propios números

```
class Numero:
    def __init__(self, valor):
        self.valor = valor

    def __add__(self, otro):
        resultado = self.valor + otro.valor
        print(self.valor, "+", otro.valor, "=", resultado)
        return resultado

    def __sub__(self, otro):
        resultado = self.valor - otro.valor
        print(self.valor, "-", otro.valor, "=", resultado)
        return resultado

    def __mul__(self, otro):
        resultado = self.valor * otro.valor
        print(self.valor, "*", otro.valor, "=", resultado)
        return resultado

    def __truediv__(self, otro):
        if(otro.valor == 0):
            print("NO PUEDO DIVIDIR POR 0")
        else:
            resultado = self.valor / otro.valor
            print(self.valor, "/", otro.valor, "=", resultado)
            return resultado

#-----

a = Numero(6)
b = Numero(15)
c = Numero(0)

resultado = a + b  #--> 6 + 15 = 21
print(resultado)  #--> 21

resultado = b - c  #--> 15 - 0 = 15
print(resultado)  #--> 15

resultado = a * b  #--> 6 * 15 = 90
print(resultado)  #--> 90

resultado = a / c  #--> NO PUEDO DIVIDIR POR 0
print(resultado)  #--> None

resultado = b / a  #--> 15 / 6 = 2.5
print(resultado)  #--> 2.5
```

Tip: TODO EN PYTHON ES UN OBJETO!

Python por debajo trata a todo, si leíste bien, A TODO tipo de dato como objeto. Los ints son objetos, los strings son objetos, las listas, sets, los float, TODO!

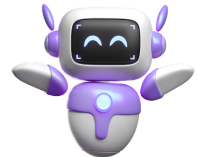
Si te das cuenta, cuando hacemos algo como: **lista.append()** estamos usando un método exclusivo (**.append(cosa)**) de la clase **list**.

Cuando hacemos algo como "Arroz" "Fideos", estamos usando el método **__lt__()** de los strings.

Y así con todo, **INCLUSO LAS FUNCIONES**, son un tipo de objeto que tiene implementado el método **__call__()**, por eso podemos hacer algo como **función(cosas)**.

Perdón si le puse mucho énfasis a esto, pero esta es de las bases para entender como funciona Python por detrás y porque a veces tiene los comportamientos que tiene :D

*(PD: si quieres investigar mas, puedes usar la funcion **dir()** de Python para saber todo lo que tienen implementado las estructuras de datos, tanto métodos atributos).*



6.5. Properties (avanzado)

Imagina que debes verificar constantemente si a un atributo o método de clase se le esta asignando un valor coherente. Veamos el siguiente ejemplo usando una clase "Persona":

```

class Persona:
    especie = "Homo Sapiens"
    def __init__(self, nombre, edad):
        self.nombre = nombre
        self.edad = edad

#-----

def crear_persona(nombre, edad):
    """
    Funcion que crea una persona y se encarga que sea valida
    Retorna: Persona(nombre, edad), si no es valida, retorna None
    """

    if(nombre == ""):
        print("Nombre no puede estar vacio")
        nombre = "vacio"

    if(edad < 0):
        print("Edad no puede ser negativa")
        edad = 0

    print("Creando persona", nombre, "de", edad, "años...")
    return Persona(nombre, edad)

#-----

persona = crear_persona("Paula", 15) #--> Creando persona Paula de 15 años...
print(persona) #--> Persona object <Paula, 15>

persona = crear_persona("", 30) #--> Nombre no puede estar vacio
print(persona) #--> Persona object <vacio, 30>

persona = crear_persona("Martin", -1) #--> Edad no puede ser negativa
print(persona) #--> Persona object <Martin, 0>

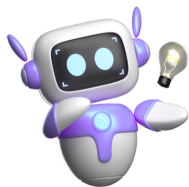
```

Setters y Getters

A esta manera de encapsular comportamiento (acceso / modificación / eliminación) se le llama **setters, getters y deleters**.

- **Setters:** Son funciones encargadas de modificar los atributos del objeto
- **Getters:** Son funciones encargadas de acceder a los atributos del objeto
- **Deleter:** Son funciones encargadas de eliminar el objeto por completo

Esta es una buena practica, ya que hace que tus objetos no pierdan encapsulan y mantiene el código legible y a prueba de errores.



Para evitar tener que crear todas estas funciones externas, que validen valores (esto rompe con lo bonito de abstraer cosas a objetos). Existe un 5.9 **decorador** llamado **@property**. El cual encapsula estas verificaciones para

que usar la clase siga siendo igual de simple.

Código con property agregado

```
class Persona:
    especie = "Homo Sapiens"
    def __init__(self, nombre, edad):
        self._nombre = ""
        self._edad = 0

        self.nombre = nombre
        self.edad = edad

        print("Creando persona", self.nombre, "de", self.edad, "años...")

    @property
    def nombre(self):
        return self._nombre

    @property
    def edad(self):
        return self._edad

    @nombre.setter
    def nombre(self, valor):
        if(valor == ""):
            valor = "vacio"
        self._nombre = valor

    @edad.setter
    def edad(self, valor):
        if(valor < 0):
            valor = 0
        self._edad = valor

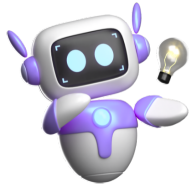
#-----

persona = Persona("Paula", 15) #--> Creando persona Paula de 15 años...
print(persona) #--> Persona object <Paula, 15>

persona = Persona("", 30) #--> Creando persona vacio de 30 años...
print(persona) #--> Persona object <vacio, 30>

persona = Persona("Martin", -1) #--> Creando persona Martin de 0 años...
print(persona) #--> Persona object <Martin, 0>
```

No es necesario que implementes siempre los 3. A veces solo basta con tener un buen **getter**. Otras con un setup de **setter + getter** y así.



Uso de @property

```
class MiClase:
    def __init__(self):
        #hacer cosas
        self._atributo = algo

    #-----

    @property
    def atributo(self):
        return self._atributo #Getter

    @atributo.setter
    def atributo(self, valor_nuevo):
        #Hacer cosas (validar, etc)

        self._atributo = valor_nuevo #Setter

    @atributo.deleter
    def atributo(self):
        del self._atributo #Deleter
```

Una convención para las Properties es usar **_atributo** para los atributos que queremos hacer **"privados"**, es decir: no queremos poder modificarlos directamente desde afuera.

En Python no existen los atributos privados, a pesar de tener **_** en el nombre, por eso esto es solo una convención a seguir entre programadores. Una buena practica es que la respetes y si ves algún método o atributo con **_** dentro de alguna clase, lo ideal es que no lo toques y uses los **getters** y **setters** que estén programados para ello.

Ejemplo: Uso de property para una tienda

```
class TiendaDeRopa:
    """
    Tienda de ropa:
    Nombre = Se muestra bonito en pantalla
    Productos = lista de listas: [[nombre, precio], [nombre_2, precio_2], etc..]
    Dinero = Si < 0 se avisa que no se puede gastar mas y se queda en -1
    """

    def __init__(self, nombre, productos):
        self.productos = productos
        self._nombre = nombre
        self._dinero = 0
        self.comision = 2000

    @property
    def nombre(self):
        return ("Tienda de ropa: " + self._nombre)

    @property
    def dinero(self):
        return self._dinero

    @dinero.setter
    def dinero(self, nuevo_valor):
        if(nuevo_valor < 0):
            print("No puedes gastar mas :(")
            nuevo_valor = -1

        self._dinero = nuevo_valor

    #-----
    def vender_producto(self, indice):
        nombre = self.productos[indice][0]
        valor = self.productos[indice][1]

        print("Vendiendo", nombre, "por $", valor)
        self.productos.pop(indice)
        self.dinero += valor

    def comprar_producto(self, producto, precio):
        print("Tratando de comprar", producto, "a $", precio)
        self.dinero -= precio

        if(self.dinero == -1):
            print("Compra fallida :(")
            self.dinero = 0
        else:
            print("Compra exitosa :D")
            self.productos.append([producto, precio + self.comision])
```

Ejemplo: Uso de property para una tienda

```
stock_inicial = [ ["Polera", 9000], ["Zapatillas", 120000], ["Pantalon", 30000] ]
tienda = TiendaDeRopa("SuperClothes", stock_inicial)

print(tienda.nombre) #--> Tienda de ropa: SuperClothes
print("Dinero restante: $", tienda.dinero) #--> Dinero restante: $0

tienda.comprar_producto("Chaqueta", 50000) #--> No puedes gastar mas :( --> Compra fallida
tienda.vender_producto(1) #--> Vendiendo Zapatillas por $120.000
tienda.vender_producto(0) #--> Vendiendo Polera por $9.000

print("Dinero restante: $", tienda.dinero) #--> Dinero restante: $129.000
tienda.comprar_producto("Chaqueta", 50000) #--> Compra exitosa :D
```

6.6. Polimorfismo (avanzado)

El polimorfismo no es mas que una definición para el hecho de que distintas clases pueden tener los mismos nombres en sus métodos, pero comportarse distinto.

Ejemplo de polimorfismo

```
class Perro:
    def __init__(self):
        pass #No hace nada

    def hacer_ruido(self):
        print("Guau guau")

#-----

class Gato:
    def __init__(self):
        pass #No hace nada

    def hacer_ruido(self):
        print("Meow")

#-----

class Noctulo:
    def __init__(self):
        pass #No hace nada

    def hacer_ruido(self):
        print("Nunca nos vemos")

#-----

perro = Perro()
gato = Gato()
noctulo = Noctulo()

perro.hacer_ruido() #--> Guau Guau
gato.hacer_ruido()  #--> Meow
noctulo.hacer_ruido() #--> Nunca nos vemos
```

6.7. Herencia (avanzado)

En Python, las clases pueden heredar comportamientos de otras clases, aumentando así la eficiencia al no tener que estar reescribiendo código una y otra vez.

La clase que hereda se llama **subclase** mientras que la clase superior a ella se llama **superclase**.

Herencia entre objetos

```
class SuperClase:
    def __init__(self, *args):
        #constructor de la superclase
        self.atributo = algo

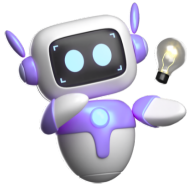
    def metodo_super_clase(self):
        #cosas

class SubClase(SuperClase):
    '''
    Clase que hereda todo lo de la SuperClase
    '''
    def __init__(self, *args):
        super().__init__(*args) #Inicializar a la superclase

        #Inicializarme yo
        self.atributo_subclase = algo

    def metodo_ampliado(self):
        #blah blah
        self.atributo = algo

        #notar que puedo usar todo lo de SuperClase
        self.metodo_super_clase()
```



Ejemplo de herencia (Vehículos)

```

class Vehiculo:
    '''
    Clase base para otros vehiculos
    '''
    def __init__(self, velocidad_max, color):
        self.velocidad_max = velocidad_max
        self.velocidad_actual = 0
        self.color = color

    def acelerar(self, aceleracion):
        self.velocidad_actual += aceleracion

    def desacelerar(self, desaceleracion):
        self.velocidad_actual -= desaceleracion

    def pisarle_a_fondo(self):
        self.velocidad_actual = self.velocidad_max

    def mostrar_velocidad(self):
        return (str(self.velocidad_actual) + " / " + str(self.velocidad_max))

#-----

class McLaren(Vehiculo):
    def __init__(self, modelo, precio, *args):
        super().__init__(*args) #Inicializar velocidad_max y color
        self.marca = "McLaren"
        self.modelo = modelo
        self.precio = precio

    def pisarle_a_fondo(self):
        super().pisarle_a_fondo()
        print("Acelerando", self.modelo, "a", self.velocidad_actual)

    def flexear(self):
        print("Soy un", self.marca, "| modelo:", self.modelo, "| color:", self.color, "| valor $", str(

class Suzuki(Vehiculo):
    def __init__(self, modelo, precio, *args):
        super().__init__(*args) #Inicializar velocidad_max y color
        self.marca = "Suzuki"
        self.modelo = modelo
        self.precio = precio

    def pisarle_a_fondo(self):
        super().pisarle_a_fondo()
        print("Acelerando", self.modelo, "a", self.velocidad_actual)

    def flexear(self):
        print("Soy un", self.marca, "| modelo:", self.modelo,\
              "| color:", self.color, "| valor $", str(self.precio))

```

```

auto_1 = McLaren("720s", 320000000, 200, "Naranja")
auto_2 = McLaren("P1", 1100000000, 350, "Amarillo")

auto_3 = Suzuki("S-Presso", 9000000, 145, "Gris")

```

Para poder acceder a los métodos / atributos de la clase de la que se esta heredando, se puede usar la función **super()**. Esta retorna a la super clase (MRO). De igual forma, si no se **sobrescribio** (override) el método o atributo. Se puede usar directamente sin el uso de **super()**. Ya que la **subclase** hereda todo lo de la **superclase**

Uso de super

```
class SuperClase:
    def __init__(self):
        pass

    def hacer_algo(self):
        print("Superclase hace algo")

class SubClase(SuperClase):
    def __init__(self, *args):
        super().__init__(*args)

    def hacer_algo(self):
        print("Subclase hace algo")

    def super_hace_algo(self):
        super().hacer_algo() #--> llamar al metodo de SuperClase

#-----

a = SuperClase()
b = SubClase()

a.hacer_algo() #--> Superclase hace algo
b.hacer_algo() #--> SubClase hace algo
b.super_hace_algo() #--> Superclase hace algo
```

Notar que también se puede heredar "infinitamente" entre clases, por ejemplo. Un **perro** hereda los atributos de un **mamífero** que hereda los atributos de un **ser vivo** y así...

Ejemplo: Cadena de herencias (Múltiples niveles de herencia)

```

class SerVivo:
    def __init__(self, altura, peso):
        self.altura = altura
        self.peso = peso

    def respirar(self):
        print("Respirando...")

    def comer(self, comida):
        print("Comiendo...", comida)

#-----

class Mamifero(SerVivo):
    def __init__(self, especie, puede_volar, **kwargs):
        super().__init__(**kwargs)
        self.especie = especie
        self.puede_volar = puede_volar

    def volar(self):
        if(self.puede_volar):
            print("Esta volando...")
        else:
            print("No sabe volar :(")

#-----

class Perro(Mamifero):
    def __init__(self, nombre, raza, altura, peso):
        super().__init__(especie="Canis familiaris", \
            puede_volar=False, altura=altura, peso=peso)

        self.nombre = nombre
        self.raza = raza
        self.presentarse()

    def ladrar(self):
        self.respirar()
        print("Guau guau")

    def comer(self, comida):
        print(self.nombre, "se prepara para comer")
        super().comer(comida)

    def presentarse(self):
        print("Este es un", self.raza, "llamado", self.nombre, "de", str(self.peso), "kg")

    def volar(self):
        print(self.nombre, "se prepara para volar...")
        super().volar()

#-----

```

```

perro = Perro(nombre="Princesa", raza="Pitbull", altura=42, peso=25)
#--> Este es un Pitbull llamado Princesa de 25 kg

```

IIC2233

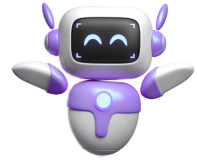
```

perro.ladrar() #--> Respirando... Guau guau
print(perro.especie) #--> Canis familiaris
perro.comer("Churu") #--> Comiendo... Churu

```

Tip: El uso de `super().__init__()`

Una buena practica es siempre usar **`super().__init__()`** en el constructor de nuestras **subclases**, ya que nos aseguramos que se inicialicen correctamente todos los métodos y atributos de la **superclase**.



También se puede heredar de múltiples clases a la vez, pero esto lo veremos mas adelante (ahora).

6.7.1. Multiherencia (avanzado)

Una clase también puede heredar cosas de muchas clases a la vez.

Multiherencia

```
class ClaseA:
    def __init__(self):
        pass

    #cosas de clases

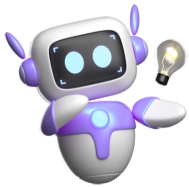
class ClaseB:
    def __init__(self):
        pass

    #cosas de clases

#-----

class ClaseC(ClaseA, ClaseB):
    """
    Esta clase heredara todo lo de A y B
    """
    def __init__(self):
        pass

    #cosas de clases
```



Ejemplo: multiherencia

```

class Persona:
    def __init__(self, nombre, edad, trabajo, **kwargs):
        self.nombre = nombre
        self.edad = edad
        self.trabajo = trabajo

    def trabajar(self):
        print(f"{self.nombre} esta trabajando de {self.trabajo}")

    def presentarse(self):
        print(f"Hola, soy {self.nombre}, tengo {self.edad} años y trabajo de {self.trabajo}")

class PersonaMagica:
    def __init__(self, poder, **kwargs):
        self.superpoder = poder

    def usar_poder(self):
        print(f"Usando {self.superpoder}...")

#-----

class SuperHeroe(Persona, PersonaMagica):
    '''
    Clase hibrida entre ambas clases
    '''

    def __init__(self, nombre, superpoder, **kwargs):
        Persona.__init__(self, nombre=nombre, edad=20, trabajo="Mesero")
        PersonaMagica.__init__(self, poder=superpoder, **kwargs)

    def presentarse(self):
        super().presentarse()
        print(f"Pero tambien tengo el superpoder de {self.superpoder}")

    def combatir(self):
        print(f"Estan atacando a {self.nombre}")
        self.usar_poder()

#-----

persona = Persona("Jesus", 20, "Quimico")
persona_2 = SuperHeroe("Batman", "Dinero")
persona_3 = SuperHeroe("Hombre invisible", "Invisibilidad")

persona.presentarse() #--> Hola, soy Jesus, tengo 20 años y trabajo de Quimico
persona.trabajar() #--> Jesus esta trabajando de Quimico

persona_2.presentarse() #--> Hola, soy Batman, tengo 20 años y trabajo de Mesero
#Pero tambien tengo el superpoder de Dinero

persona_2.trabajar() #--> Batman esta trabajando de Mesero

persona_3.presentarse() #--> Hola, soy Hombre invisible, tengo 20 años y trabajo de Mesero
#Pero tambien tengo el superpoder de Invisibilidad

persona_3.combatir() #--> Estan atacando a Hombre invisible
#Usando Invisibilidad...

```

Notar que no es necesario reimplementar todos los métodos de la **super clase** en la **subclase**, sin embargo es posible sobrescribirlos (override) para obtener nuevas funcionalidades si así se desea.

Pero aquí pueden ocurrir varios problemas si se mezcla con el 6.6 **Polimorfismo**, que pasa si quiero llamar a un método que se llama igual entre las clases de las que heredo? (a quien llamo?).

Polimorfismo y Multiherencia

```
class A:
    def __init__(self):
        print("Iniciando A...")

    def hacer_cosas(self):
        print("A esta haciendo cosas..")

class B(A):
    def __init__(self):
        print("Iniciando B...")

    def hacer_cosas(self):
        print("B esta haciendo cosas..")

class C(A):
    def __init__(self):
        print("Iniciando C...")

    def hacer_cosas(self):
        print("C esta haciendo cosas..")

class D(B, C):
    def __init__(self):
        print("Iniciando D...")

#-----

cosa = D()
cosa.hacer_cosas() #A quien llamo (B / C / A) ?
```

Para establecer a que método en particular se llama existe el **Method Resolution Order** (MRO). El cual no es mas que una lista que Python sigue a la hora de ver de que clase llamar los métodos y al usar la función súper.

Imaginalo como una lista de prioridades que Python tiene por debajo y sigue a la hora de resolver de cual de todas las clases en la linea hereditaria llamar el método que estamos pidiendo. Esto sirve para resolver el siguiente problema que veremos en esta guía **el problema del diamante en Multiherencia**.

MRO de las clases del ejemplo anterior

```
'''
Se puede ver de 2 formas (son lo mismo):

    Nombreclase.mro()
    Nombreclase.__mro__
'''

print(A.mro())#--> [clase <A>]

print(B.__mro__) #--> [clase <B>, clase <A>]

print(C.mro()) #--> [clase <C>, clase <A>]

print(D.__mro__) #--> [clase <D>, clase <B>, clase <C>, clase <A>]

'''
La gracia es que Python seguira esta lista en orden y
usara a la primera clase que tenga el metodo implementado
'''
```

6.7.2. El problema del diamante en Multiherencia

Imagina que tenemos esta linea hereditaria de clases:

```
class A:
    def __init__(self):
        print("Inicializando A...")

    def hacer_cosas(self):
        print("A esta haciendo cosas..")

class B(A):
    def __init__(self):
        print("Inicializando B...")

    def hacer_cosas(self):
        print("B esta haciendo cosas..")
        A.hacer_cosas(self)

class C(A):
    def __init__(self):
        print("Inicializando C...")

    def hacer_cosas(self):
        print("C esta haciendo cosas..")
        A.hacer_cosas(self)

class D(B, C):
    def __init__(self):
        print("Inicializando D...")

    def hacer_cosas(self):
        print("D esta haciendo cosas..")
        B.hacer_cosas(self)
        C.hacer_cosas(self)

#-----

cosa = D()
cosa.hacer_cosas()

'''
OUTPUT:

D esta haciendo cosas..

B esta haciendo cosas..
A esta haciendo cosas..

C esta haciendo cosas..
A esta haciendo cosas..
'''
```

Notemos que si llamamos al método **hacer_cosas()** en **D**, pasara lo siguiente:

- **Clase D:** Hara cosas y llamara al método de **hacer_cosas()** de **B Y C**.
- **Clase B:** Hara cosas y llamara al método de **hacer_cosas()** de **A**.
- **Clase C:** Hara cosas y llamara al método de **hacer_cosas()** de **A** (de nuevo)

Notas que se llama 2 veces al método **hacer_cosas()** de **A**?. Esto se traduce generalmente en comportamientos inesperados, menor eficiencia en el código, etc.

A esto, se le llama problema del diamante por que si vemos la forma que se hace en el **diagrama de clases** esta es parecida a un diamante (**1 clase hereda de 2 clases que heredan de una misma clase**).

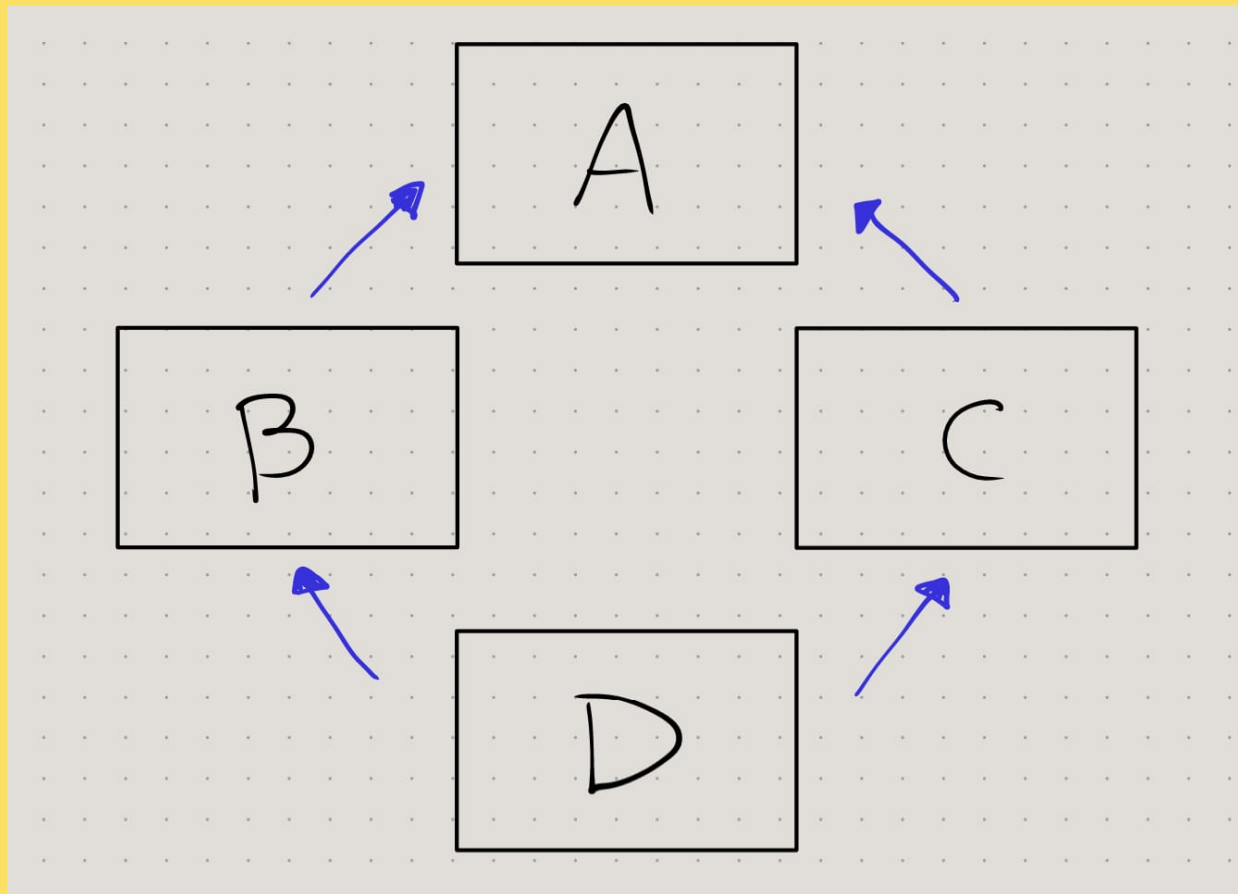


Figura 18: Diagrama de clases de ejemplo anterior

Este problema se puede resolver fácilmente usando la función **super()** anteriormente mencionada, ya que esta se guía por el 6.7.1 **MRO**, el cual se encarga de que no se repitan llamadas a métodos entre clases (linealiza la cadena de herencia).

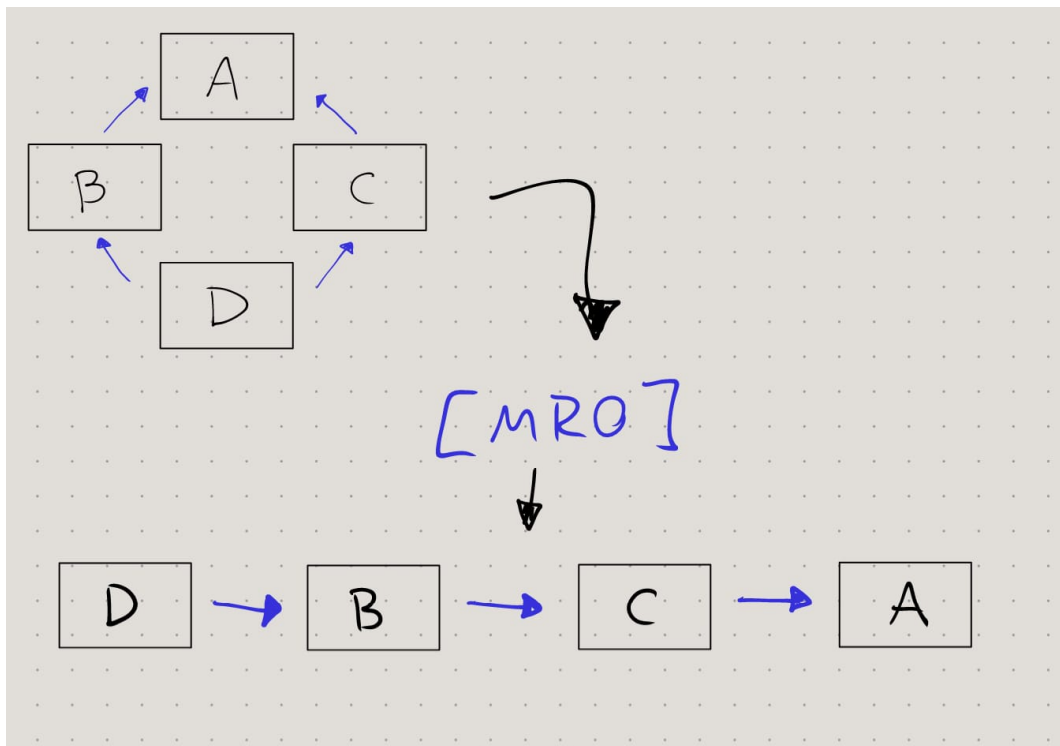


Figura 19: Ejemplo de como `super()` usa MRO para linealizar la cadena de herencia

Ejemplo: uso de super para aliviar el problema del diamante anterior

```
class A:
    def __init__(self):
        print("Inicializando A...")

    def hacer_cosas(self):
        print("A esta haciendo cosas..")

class B(A):
    def __init__(self):
        print("Inicializando B...")

    def hacer_cosas(self):
        print("B esta haciendo cosas..")
        super().hacer_cosas()

class C(A):
    def __init__(self):
        print("Inicializando C...")

    def hacer_cosas(self):
        print("C esta haciendo cosas..")
        super().hacer_cosas()

class D(B, C):
    def __init__(self):
        print("Inicializando D...")

    def hacer_cosas(self):
        print("D esta haciendo cosas..")
        super().hacer_cosas()

#-----

cosa = D()
cosa.hacer_cosas()

'''
OUTPUT:

D esta haciendo cosas..
B esta haciendo cosas..
C esta haciendo cosas..
A esta haciendo cosas..
'''
```

6.8. Clases abstractas

Ya vimos ejemplos donde la herencia entre clases funciona muy bien para denotar comportamientos que se repiten entre clases. Por ejemplo: Podemos tener una clase base que se llame **Figura**, de la cual hereden sub clases

como: **Circulo**, **Triangulo**, **Cuadrado**, etc..

Ejemplo de clases que tienen comportamientos similares

```
class Figura:
    def __init__(self, nombre):
        self.nombre = nombre

    def dibujar(self):
        pass

    def calcular_area(self):
        pass

#=====

class Triangulo(Figura):
    def __init__(self, base, altura):
        super().__init__("Triángulo")
        self.base = base
        self.altura = altura

    def dibujar(self):
        print("Dibujando triangulo")

class Cuadrado(Figura):
    def __init__(self, lado):
        super().__init__("Cuadrado")
        self.lado = lado

    def dibujar(self):
        print("Dibujando cuadrado")
```

Sin embargo, que pasa si nuestras sub clases no tienen sobre escrito alguno de los metodos en **Figura**?

Otro caso, que sucederia si no queremos que se pueda instanciar directamente una **Figura**, ya que, no tiene ningun sentido logico tener una figura sin nombre o que no sepamos como se dibuja.

Para estos casos, podria decirse que queremos que **Figura** sea una "plantilla" para otras clases. A esta "plantilla" se le llama: **Clase abstracta** y su funcionalidad es justamente servir como una plantilla para otras clases y no poder ser directamente instanciada!

6.9. Utilizacion de clases abstractas (modulo abc)

Para crear una clase abstracta, necesitaremos del modulo **abc** de Python, de este modulo necesitaremos 2 cosas:

- **ABC**: Clase base de la que heredara nuestra clase abstracta (tiene los handlers para que efectivamente nuestra clase sea abstracta.
- **abstractmethod**: Decorador que nos dejara crear nuestros metodos abstractos (metodos que queremos que se sobrescriban en nuestras sub clases derivadas)

Implementacion de una clase abstracta

Ejemplo de clase plantilla

```
from abc import ABC, abstractmethod

class Figura(ABC):
    def __init__(self, nombre):
        self.nombre = nombre

    @abstractmethod
    def dibujar(self):
        pass

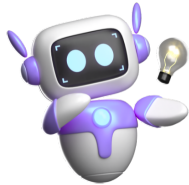
    @abstractmethod
    def calcular_area(self):
        pass
```

Ejemplo de subclase

```
class Triangulo(Figura):
    def __init__(self, base, altura):
        super().__init__("Triángulo")
        self.base = base
        self.altura = altura

    def dibujar(self):
        print("Dibujando triangulo")

    def calcular_area(self):
        return (self.base * self.altura) / 2
```



Ahora no podremos hacer una instancia de `Figura` directamente y todas las clases derivadas de nuestra clase abstracta DEBEN sobrescribir nuestro metodo abstracto **dibujar**, de lo contrario Python arrojara error. Justo el comportamiento que queriamos!

Intento de instanciar una clase abstracta

```
figura_1 = Figura("Circulo")
```

#ERROR: No puedes instanciar una clase abstracta wachin

Intento de instanciar una subclase sin metodos sobrescritos

```
from abc import ABC, abstractmethod

class Figura(ABC):
    def __init__(self, nombre):
        self.nombre = nombre

    @abstractmethod
    def dibujar(self):
        pass

#=====

class Circulo(Figura):
    def __init__(self):
        super().__init__("Circulo")

    #Falta sobrescribir dibujar()

#=====

figura_2 = Circulo()

#ERROR: No sobrescribiste todos los metodos que requiere tu subclase
```

En el momento en que definimos algun metodo abstracto ahi recien nuestra clase sera abstracta, el hecho de heredar de ABC no tiene ningun efecto por si solo mas que de agregar la posibilidad de usar @abstractmethod.

Clase que hereda de ABC pero le falta decorador abstract

```
from abc import ABC, abstractmethod

class Figura(ABC):
    def __init__(self, nombre):
        self.nombre = nombre

    def dibujar(self):
        pass

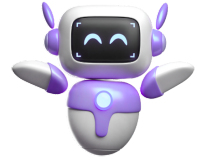
    def calcular_area(self):
        pass

#=====
#Notar que podemos usarla sin problemas, ya que aun no se vuelve abstracta
#(falta el decorador abstract en alguno de sus metodos)

figura_1 = Figura("Platano")
```

Tip: Cuando utilizar una clase abstracta

Como vimos anteriormente, el mejor uso de una clase abstracta es cuando queramos hacer muchas subclases que deriven de una misma "plantilla" sin embargo queremos que esta ultima se mantenga como una plantilla y no pueda ser instanciada :p



6.10. Resumen de OOP

6.10.1. Cuando usarlo?

La **Programación orientada a objetos (OOP)** organiza el código en torno a objetos que modelan algún comportamiento (real o abstracto), encapsulando estos comportamientos propios a métodos y atributos fáciles de usar.

Úsalo cuando tengas que simular / modelar situaciones que dependan de muchas variables a la vez o que hayan muchas variables que cambien según comportamientos específicos/situacionales.

6.10.2. Clases

Las clases son **plantillas** para los objetos mientras que las **instancias** son los objetos en concreto.

Estructura básica de una clase

```
class Nombre_clase:
    """
    Aqui puedes colocar todos los atributos / variables
    que creas que necesita tu clase
    """

    #Inicializador / Constructor
    def __init__(self):
        #Esta sera llamada cada vez que instanciemos
        #Alguno de estos objetos

    """
    Aqui ya puedes agregarle lo que tu quieras
    que tenga (metodos / atributos)
    """

    def metodo(self, a, b, c, etc...):
        #cosas
```

La palabra **self** debe ser usada como primer parámetro en todos sus métodos

Dunder methods

Al sobrescribir los métodos **__método__()** cambias el comportamiento de base que tiene Python con tus objetos, ya sea comparándolos, operándolos entre sí, etc.

6.10.3. Instancias

Las instancias son los objetos en concreto, cada una es totalmente independiente de la otra y posee una identidad propia dentro del código.

Para acceder a sus métodos o atributos basta con usar la notación **objeto.atributo** / **objeto.metodo()**.

Uso de instancias

```
instancia = Clase(a, b, c, d...)

instancia.atributo #--> Acceder a un atributo

instancia.metodo() #--> Usar un metodo de la clase
```

6.10.4. Herencia (avanzado)

Permite crear nuevas **subclases** reutilizando métodos o atributos de una **superclase**, se puede tener una **cadena de herencias** ($A \Rightarrow B \Rightarrow C \Rightarrow \text{etc.}$) o bien heredar de varias **superclases** a la vez.

Herencia

```
class A:
    #cosas de clase

    def metodo():
        #blah blah

class B(A):
    #cosas de subclase de A

class C(A):
    #cosas de subclase de A

    def metodo():
        #blah blah

    #super(): Si tengo sobrescrito lo de una clase
    #superior pero quiero usarlo igualmente

    self.metodo() #--> Me llamara a mi version sobrescrita
    super().metodo() #--> Buscara con el MRO la version superior a usar

#-----
#Mutliherencia

class D(B, C):
    #cosas de subclase hibrida entre B y C

    #super(): para usar MRO y solucionar problema del
    #diamante en caso de que ambas superclases hereden de la misma clase
```

Con la **multiherencia** se debe tener cuidado con el 6.7.2 **problema del diamante**

La funcion **super()** ayuda a manejar de mejor manera tanto el problema del diamante, como especificar a cual de las clases en la cadena hereditaria se esta refiriendo el código.

6.10.5. Clases abstractas (modulo abc)

Permite crear "plantillas" para futuras clases y asi ahorrarnos bastante tiempo codeando y tambien se encarga de que nuestras subclases sigan una estructura deseada.

Clases abstractas

```
from abc import ABC, abstractmethod

class Plantilla(ABC):
    def __init__(self, *args, **kwargs):
        #hacer cosas

    @abstractmethod
    def metodo_abstracto(self):
        #subclases necesitan sobrescribirlo
        #sino codigo arroja error
```

6.11. Ejercicios de práctica

Reto: Clase básica

Crea una clase llamada **Persona** con atributos **nombre** y **edad**, y un método para saludar que imprima "Hola, soy nombre".

Ejemplo 1

Input

No aplica

Output

Hola, soy Juan

Reto: Constructor y atributos

Modifica la clase `Persona` para que su constructor reciba **nombre** y **edad**, y crea un método que devuelva la **edad**.

Ejemplo 1

Input

No aplica

Output

30

Reto: Métodos especiales

Agrega un método `__str__` a la clase `Persona` que retorne "Nombre: **nombre**, Edad: **edad**".

Ejemplo 1

Input

No aplica

Output

Nombre: Ana, Edad: 25

Reto: Herencia

Crea una clase `Estudiante` que herede de `Persona` y añada un atributo **curso**. Añade un método que imprima "Estudio en **curso**".

Ejemplo 1

Input

No aplica

Output

Estudio en Ingeniería

Reto: Encapsulamiento

Haz que el atributo **edad** sea privado y crea métodos getter y setter para acceder y modificar su valor.

6.12. Soluciones a los ejercicios de práctica

Solucion: Clase básica

Definimos la clase con atributos y método para saludar.

```
class Persona:
    def __init__(self):
        self.nombre = "Juan"
        self.edad = 30

    def saludar(self):
        print("Hola, soy " + self.nombre)

#-----
#Ejemplo de uso
persona1 = Persona()
persona1.saludar()
```

Solucion: Constructor y atributos

Implementamos el constructor `__init__` y método para obtener la edad.

```
class Persona:
    def __init__(self, nombre, edad):
        self.nombre = nombre
        self.edad = edad

    def saludar(self):
        print("Hola, soy " + self.nombre)

    def obtener_edad(self):
        return self.edad

#-----
#Ejemplo de uso
persona1 = Persona("Juan", 30)
print(persona1.obtener_edad())
```

Solucion: Métodos especiales

Definimos el método `__str__` para mostrar una representación amigable del objeto.

```
class Persona:
    def __init__(self, nombre, edad):
        self.nombre = nombre
        self.edad = edad

    def saludar(self):
        print("Hola, soy " + self.nombre)

    def obtener_edad(self):
        return self.edad

    def __str__(self):
        return "Nombre: " + self.nombre + ", Edad: " + str(self.edad)

#-----
#Ejemplo de uso
persona1 = Persona("Juan", 30)
print(persona1)
```

Solucion: Herencia

Creamos la subclase `Estudiante` que hereda de `Persona` y añadimos atributos y métodos mediante `super()` y `**kwargs`.

```
class Persona:
    def __init__(self, *args, **kwargs):
        self.nombre = kwargs['nombre']
        self.edad = kwargs['edad']

    def saludar(self):
        print("Hola, soy " + self.nombre)

    def obtener_edad(self):
        return self.edad

    def __str__(self):
        return "Nombre: " + self.nombre + ", Edad: " + str(self.edad)

class Estudiante(Persona):
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.curso = kwargs['curso']

    def describir_carrera(self):
        print("Estudio en " + self.curso)

#-----
#Ejemplo de uso
persona1 = Estudiante(nombre="Juan", edad=30, curso="Ingenieria")
persona1.describir_carrera()
```

Solucion: Encapsulamiento

Se utiliza el prefijo `__` para atributos privados y métodos getter y setter para acceder a ellos.

```
class Persona:
    def __init__(self, *args, **kwargs):
        self.nombre = kwargs['nombre']
        self._edad = kwargs['edad']

    def saludar(self):
        print("Hola, soy " + self.nombre)

    @property
    def edad(self):
        return self._edad

    @edad.setter
    def edad(self, nueva_edad):
        self._edad = nueva_edad

    def __str__(self):
        return "Nombre: " + self.nombre + ", Edad: " + str(self.edad)

class Estudiante(Persona):
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.curso = kwargs['curso']

    def describir_carrera(self):
        print("Estudio en" + self.curso)

#-----
#Ejemplo de uso
persona1 = Estudiante(nombre="Juan", edad=30, curso="Ingenieria")

persona1.edad += 1 #Juan esta de cumpleaños!
print(persona1)
```